Empirical Software Engineering (2025) 30:95
https://doi.org/10.1007/s10664-025-10646-w

A systematic review on smart contracts security design patterns

Sadaf Azimi[1] · **Ali Golzari**[1] · **Naghmeh Ivaki**[1] · **Nuno Laranjeiro**[1]

Accepted: 21 March 2025 / Published online: 8 April 2025
© The Author(s) 2025

Abstract
Smart contracts have accelerated the adoption of blockchain technology across various domains by enabling coded agreements between transaction participants. However, increased software defects and vulnerabilities in smart contracts, driven by developer inexperience with languages like Solidity and a lack of effective detection tools, pose significant risks. Given the high value of assets managed on blockchain (e.g., cryptocurrencies), these vulnerabilities can lead to severe consequences. Researchers and practitioners have proposed numerous smart contract design patterns to mitigate certain faults or vulnerabilities. Despite these efforts, it remains unclear which types of defects these patterns target and how effectively they address the wide range of existing smart contract security vulnerabilities. In this paper, we review the state of the art in smart contract design patterns, categorizing them and analyzing their effectiveness in mitigating known security vulnerabilities. Our findings reveal that only five patterns directly aim to prevent security vulnerabilities, collectively addressing just 6 out of 94 security issues identified by OpenSCV (a state-of-the-art vulnerability taxonomy), highlighting the need for further research on smart contract security design patterns.

Keywords Blockchain · Smart contracts · Design patterns · Security design patterns · Vulnerabilities

Communicated by: Xin Xia

✉ Sadaf Azimi
sadafazimi@dei.uc.pt

Ali Golzari
aligolzari@dei.uc.pt

Naghmeh Ivaki
naghmeh@dei.uc.pt

Nuno Laranjeiro
cnl@dei.uc.pt

1 CISUC/LASI - Centre for Informatics and Systems of the University of Coimbra, Department of Informatics Engineering, University of Coimbra, Coimbra, Portugal

1 Introduction

Blockchain systems are nowadays being used in various domains, including finance, supply-chain management, and healthcare, to name a few. The increased adoption of this kind of system is linked to the advances in smart contract technology, which has become a quite popular way of developing blockchain applications. As in other types of software, and also due to being a relatively recent way of developing programs, smart contracts are prone to carry residual code defects, including software vulnerabilities (Vidal et al. 2024).

The presence of a security vulnerability in a smart contract has consequences that are coupled with the domain of the system and the specificities of the technology. At first, it is very likely that the activation of a vulnerability brings in financial costs due to the usual context of the use of smart contracts (Dika and Nowostawski 2018). In addition to this, in a blockchain environment, a fault or vulnerability detected in a smart contract cannot be corrected. The contract must be terminated, and a new one must be deployed instead (Chen et al. 2022), which is an additional cost (also reputational) for the application provider and emphasizes the importance of following best practices and designs for smart contract programming to prevent vulnerabilities (Singh et al. 2020).

Design patterns are established solutions for known problems, and developers are generally aware that certain issues are better dealt with in the design phase of the software. Driven by the increasing use of smart contracts and the identification of known problems, including the creation of insecure code, various types of design patterns started being proposed in the literature (Wöhrer and Zdun 2018; Kannengießer et al. 2022; Liu et al. 2018). Despite the fact that smart contract technology is relatively recent, there is already a relatively large number of works proposing design patterns of various types, including several patterns that represent the same concept but have different designations and patterns that, despite having the same name, represent distinct concepts.

In this paper, we review the state of the art in smart contract design patterns with the goal of consolidating names, pattern meanings, and types of patterns. We place special focus on security design patterns and aim to determine how well these patterns (that represent state-of-the-art design practices for creating secure smart contracts) can be mapped to the avoidance of known vulnerabilities (identified in OpenSCV Vidal et al. 2024) and can, in this sense, be used to avoid or minimize the risk of creating smart contracts holding security vulnerabilities.

We identified 144 smart contract design patterns and categorized them into four main groups: Control, Maintainability, Performance, and Security, offering a structured framework for understanding critical aspects of smart contract development. We then specifically focused on security design patterns, identifying 36 unique patterns. These patterns were categorized into 10 subcategories based on three primary concerns: (i) the smart contract vulnerabilities that can be mitigated by design patterns (according to OpenSCV classification Vidal et al. 2024), (ii) security measures that can be implemented using these patterns (e.g., integrity), and (iii) strategies for mitigating security risks through these design patterns.

Our results (available at Sadaf et al. 2024) reveal that the majority of patterns (51%) fall under the Control category, which gathers patterns that mainly address core aspects related to smart contract functionality. Security and Performance patterns are almost evenly distributed, each representing about 24% and 20%, respectively, while Maintainability patterns are the least common (5%). The largest subcategory within Performance is the Gas Consumption subcategory, with 27 patterns emphasizing the importance of efficient gas management. Interaction patterns in the Control category are the next most significant, followed by Access

Control in Security, underlining the critical need for secure access management in smart contracts. Risk Mitigation and Secure External Calls are the following important security subcategories, with 5 and 4 patterns, respectively, highlighting the importance of reducing risks and ensuring safe external interactions.

Of the security patterns identified, only five specifically target known smart contract vulnerabilities, collectively addressing just 6 out of 94 vulnerabilities in OpenSCV. This reveals a significant gap in design patterns available to help developers mitigate a broader range of security risks in smart contracts.

This paper is organized as follows. In Section 2, we describe the methodology followed for performing this systematic review. Section 3 presents the identified design patterns for smart contracts and their general categorization. Section 4 presents the security design patterns and their categorization. Section 5 presents the mapping between the security design patterns and smart contract security vulnerabilities. Section 6 highlights the main findings of this study. Finally, Section 7 concludes this paper.

2 Methodology

This section presents the procedure used to conduct the systematic literature review on smart contract security design patterns, according to established guidelines for conducting systematic reviews (Kitchenham 2004). The procedure includes the following steps:

- **Identification and analysis of related surveys**: The initial phase of this study involves the identification and analysis of secondary studies, including surveys, systematic reviews, and mapping studies, that address the topic of smart contract security design patterns. The goal is to identify any gaps or limitations in the existing literature in order to address them in our own work.
- **Definition of research questions**: Building upon the gaps identified in the previous step, we formulate a set of research questions that delineate the objectives of our systematic review.
- **Identification of primary studies**: To identify relevant primary studies that contribute to the review, we determine the appropriate data sources (e.g., search engines) and define a query composed of key terms that are relevant for answering the research questions.
- **Study selection**: In this step, we apply a set of inclusion and exclusion criteria to the primary studies identified, as well as quality assessment criteria. As the search query may return results that are not relevant to our goals, this step aims to reach a significant set of papers as a way to assure the validity of our study.
- **Data extraction and synthesis**: During this step, we extract and synthesize data from the selected primary studies to allow answering research questions.

The following subsections describe each of the procedure's steps in further detail and provide an overview of the reviewing process's outcome.

2.1 Identification and Analysis of Related Surveys

For the identification of both survey and primary papers, we relied on *Google Scholar* search engine, given its coverage of papers from well-known databases such as *IEEE Xplore*, *ACM Digital Library*, and *Science Direct*. We found this to be a reasonable balance between completeness and the required effort to compile results. We conducted the search on February

2023 and used the following query string, built based on our goals and key terms observed during the related reviews analysis:

(blockchain OR Ethereum OR "smart contract") AND ("security patterns" OR "software patterns" OR "design patterns" OR "architectural patterns" OR "security software patterns" OR "security design patterns" OR "security architectural patterns")

After analyzing the search results, we have identified 3 secondary studies (i.e., other reviews) related to this paper's topic. We list these studies below and present an analysis of them in the following paragraphs.

- Six et al. (2022b) Blockchain software patterns for the design of decentralized applications: a systematic literature review. vol 3, p 10006. https://doi.org/10.1016/j.bcra.2022.100061
- Lu et al. (2022) Patterns for blockchain-based payment applications. In: Proceedings of the 26th european conference on pattern languages of programs, Association for Computing Machinery, New York, NY, USA, EuroPLoP '21. https://doi.org/10.1145/3489449.3490006
- Kannengießer et al. (2022) Challenges and common solutions in smart contract development. IEEE Trans Softw Eng 48(11):4291–4318. https://doi.org/10.1109/TSE.2021.3116808

Six et al. (2022b) built a taxonomy of 120 blockchain patterns that consist of two meta-categories: "On-chain pattern" and "On/off-chain interaction pattern," and 15 subcategories. The On/off-chain interaction meta-category includes 86 patterns within subcategories such as Data Exchange, Data Management, Wallet and Keys, and Transaction patterns, while the On-chain meta-category includes 18 patterns within subcategories such as Domain-based patterns, Smart Contract patterns, and Data Management patterns.

Among these subcategories, Smart contract patterns include 11 security patterns, designed to address various aspects of smart contract security, such as access restriction, function execution protection, and risk mitigation. Examples of security patterns included in this subcategory are the Termination pattern, the Emergency Stop pattern, and the Check-Effects-Interaction pattern.

The paper also emphasizes the importance of software patterns in ensuring the quality and efficiency of decentralized applications. It suggests that further research could explore the translation of traditional Gang of Four (GoF) patterns (Hunt and Hunt 2013) into blockchain contexts and the development of architectural patterns and idioms for various blockchain protocols.

However, the number of security patterns included in the paper is limited, and the paper does not sufficiently address the dynamic nature of security threats in the blockchain domain. As blockchain technology evolves, new vulnerabilities and attack vectors emerge, necessitating updates and adaptations to security patterns. The article also notes that the "Smart contract pattern" subcategory is dedicated to patterns connecting to smart contract implementation and security. However, our analysis reveals that security design patterns also exist in other subcategories, such as "Data management" (e.g., Off-chain data storage, Off-chain Signatures). This discrepancy reveals an oversight in the article, suggesting a broader distribution of security design patterns across the taxonomy than initially presented.

Lu et al. (2022) reviews design patterns in blockchain-based payment applications. They present 12 patterns that address essential aspects of state transitions of tokens in blockchain-based payment applications. These patterns are categorized into two categories: token design

and payment management. The authors consider the token lifecycle, covering stages such as issuance and transfer, and associate these with the corresponding patterns. They also highlight security design patterns, such as the Multi-Signature and Escrow patterns, which provide security features to transactions. The authors propose that future research should refine the decision-making framework for pattern selection in blockchain payment applications. Despite its valuable insights into payment applications, the article falls short by not thoroughly addressing security concerns.

Kannengießer et al. (2022) conducted an analysis of challenges in smart contract development and synthesized solutions across three Distributed Ledger Technology (DLT) platforms, including Ethereum, EOSIO, and Hyperledger Fabric. These solutions include 20 software design patterns (SDPs) that aid developers, which are categorized into architectural patterns, design patterns, and idioms. Among the 20 SDPs, some specifically address security issues, such as the Checks-Effects-Interactions Pattern, which mitigates reentrancy attacks, a known security vulnerability. They also established a taxonomy of challenges and solutions, offering a structured approach to understanding and applying the SDPs. The main gap in the article is the absence of a focused discussion on security concerns, such as potential vulnerabilities and the design patterns that could safeguard against them. Furthermore, the lack of uniformity in smart contract interfaces and the need for synchronization mechanisms to prevent transaction-ordering vulnerabilities highlight the necessity for more standardized security practices.

As blockchain continues to advance, new vulnerabilities emerge, requiring not only a deeper understanding of these new vulnerabilities but also the development of security patterns preventing them. Current literature, including the analyzed papers, often lacks a dedicated focus on design patterns with security concerns. Additionally, the lack of a dedicated taxonomy that focuses only on security design patterns for smart contracts would make it challenging to identify the existing gaps and needs for new design patterns. Addressing these gaps and providing a mapping between the common security issues in smart contracts and corresponding design patterns to prevent them is essential not only for advancing the theoretical framework of blockchain but also for ensuring its practical security in real-world applications. Thus, in this study, we aim to systematically identify and review the existing smart contract design patterns, analyze the security design patterns, and provide a mapping between these patterns and the security vulnerabilities they address.

2.2 Research Questions

Based on the analysis of existing secondary studies, we defined the following set of three research questions that form the objectives of this systematic literature review:

- **RQ-1**: What smart contract design patterns have been proposed in the literature, and what are their goals and characteristics?
- **RQ-2**: Which smart contract design patterns target security, and how do they avoid or mitigate security threats?
- **RQ-3**: Which smart contracts' known vulnerabilities are directly addressed by the identified security design patterns?

RQ-1 aims to establish a catalog of smart contract design patterns, including patterns belonging to different categories (e.g., performance, security) and with different goals (e.g., transaction management, gas consumption).

Building on the outcome of RQ-1, **RQ-2** narrows the focus from the general design patterns to those with explicit security purposes and their characterization.

Finally, **RQ-3** aims to understand the exact security problems that can be resolved with each security design pattern and to create a mapping between the patterns and known vulnerabilities according to the OpenSCV smart contract vulnerability classification (Vidal et al. 2024).

2.3 Study Identification and Selection

For the identification of primary studies, we followed the same process as for secondary studies and used the same query. The following criteria were explicitly defined to minimize the erroneous inclusion or exclusion of certain papers and also to foster the reproducibility of the study selection process.

Inclusion criteria

- The work is centered around the proposal of smart contract design patterns.
- The work contributes to answering our research questions. In practice, one or more patterns should be proposed, a 'security' label should be included in one or more of the patterns, and the pattern description should provide sufficient detail to enable understanding and a possible connection to an existing vulnerability.

Exclusion criteria

- The paper lies outside the software engineering and blockchain domains.
- The paper has not been peer-reviewed or is not written in English.

As we can see in Fig. 1, our initial search resulted in 977 studies. We began by removing duplicates, which reduced the papers to 917. *To apply the inclusion and exclusion criteria*, we then went through three pruning steps, which respectively targeted the title, abstract, and full text of each study. At each step, the goal is to understand if there is reasonable evidence that we may find descriptions of smart contract security design patterns in the paper. At the title screening level, we search for such evidence in a relaxed manner, as many times, the titles do not accurately represent the content of the paper, and we are not very interested in discarding papers at this step, even if it involves additional work in the following step. At the abstract screening level, we search for reasonable evidence that the paper will fit our criteria (again, at this stage, it may not be possible to understand if a given paper is relevant or not just by examining the abstract). At the full-text screening, we determine if the paper complies with the criteria in a strict way. This means that the paper must list and describe smart contract security design patterns. The pattern descriptions must allow us to understand fundamental aspects, namely how the use of the patterns avoids or mitigates security threats (and what these threats are). In the end, if the paper complies with these criteria, it is considered part of our dataset; if it does not comply, it is excluded from the dataset.

The paper screening phase involves some subjectiveness, which should be reduced by adhering to the practices specified by the systematic literature review process (Kitchenham 2004). In our case, we took additional measures during the screening phase. Specifically, a team of three researchers was involved: two early-stage researchers were responsible for analyzing and labeling the papers at each screening step, and an experienced researcher was responsible for breaking ties and overseeing the whole screening process.

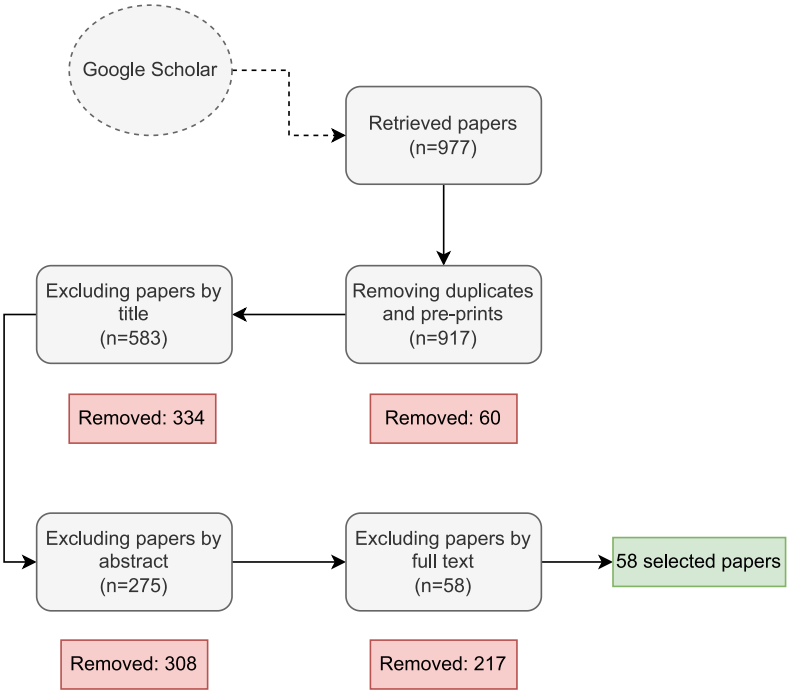

Fig. 1 Review process and outcome

The two early-stage researchers were responsible for independently marking the papers with 'yes' for those clearly relevant, 'no' or 'doubtful' for those requiring further evaluation. At all steps, we only discarded the papers where both researchers marked with a 'no', with the remaining following through for the next step. At each step, in the case of an individual doubt, we optimistically assume that the paper is related even though this leads to additional analysis effort. At the final step (i.e., the full-text analysis), the cases where researchers disagreed were individually analyzed until a consensus was reached. Whenever consensus was not attained, an experienced researcher reviewed the paper and helped in reaching the final decision. We ended up with a list of 58 papers, which are at the core of the analysis in this work.

2.4 Data Extraction and Synthesis

We identified the following three aspects as key ground **data to extract** from the studies, namely the design pattern name, the problem or challenge targeted by the pattern, and a textual description of the solution to the problem. We also collected the platform and the language to which each pattern applies. However, in a preliminary pass, we found this information to be very scarce, and therefore, we discarded it. Based on the collected ground data, we then proceeded to **synthesize information** necessary to answer our research questions and began by consolidating pattern names.

This consolidation generally consisted of identifying patterns that are essentially the same but have been proposed in the literature with different names. In general, we follow one of

the following three cases: i) We use the original name if it appears in only one paper or if it is used consistently in multiple papers; ii) We choose the most widely used name when several alternatives exist (provided one name is used more frequently); and iii) We assign a new name when name heterogeneity is high. When assigning a new name, we select one that is clearer, occasionally making adjustments for clarity or simplicity. The mapping between proposed names in the literature and our new or selected names can be found at Sadaf et al. (2024).

This process was also carried out by two early-stage researchers, who split the work. In practice, generally accepted terms (or the majority present in the papers) were selected. The outcome was reviewed by an experienced researcher.

Starting from the different patterns, we then created a structure, bottom-up, to represent the various types of patterns found. Two early-stage researchers, by consensus, proposed a layer of categories for the patterns, grouping them by similarity. These layers were then reviewed by one experienced researcher. To capture the more general intent of the patterns, an additional higher-level layer of categories was then proposed by consensus by the two early-stage researchers. These were then reviewed and discussed with one experienced researcher. The overall structure was reviewed and slightly adjusted by one additional experienced researcher. By the end of this process, we had sufficient information to answer RQ-1 and RQ-2, which, in summary, aim at identifying general-purpose design patterns and security design patterns, respectively.

The process continued and involved mapping to known smart contract security vulnerabilities. The goal would be to understand how well current design patterns cover known vulnerabilities. For this purpose, we resort to the information in OpenSCV (Vidal et al. 2024), which is a taxonomy of smart contract vulnerabilities that is more up-to-date and more comprehensive than popular alternatives such as DASP (Wong and Hemmel 2018) or SWC (Songsom et al. 2022), or even recent proposals such as Rameder et al. (2022).

2.5 Study Outcome and Comparison with Other Secondary Studies

The research was conducted in February 2023, and our initial data source, Google Scholar, yielded 977 results. Subsequent searches in the other sources mostly yielded overlapping results with Google Scholar. To simplify the presentation and avoid redundancy, we focus on reporting the outcomes of the study identification and selection process, specifically on Google Scholar.

Table 1 sets the outcome of the selection and identification process in relation to what is analyzed in related surveys. On the left-hand side, the table identifies the related surveys, the respective periods analyzed in each survey, and the total number of papers related to smart contract design patterns. On the right-hand side, the table shows the number of papers related

Table 1 Number of works analyzed in this study compared to the related reviews

	Period	Total Papers Considered	# of papers not considered		
			Inside period	Outside period	Total
Six et al. (2022b)	2017-2021	26	43	11	54
Lu et al. (2022)	2017-2021	13	41	11	52
Kannengießer et al. (2022)	2007-2021	25	43	11	54
Our work	**2017-2023**	**58**	–	–	–

to design patterns that were not considered for analysis by the authors of each survey (and are included in our own survey) within each period of analysis, outside each period of analysis, and a total count.

Table 2 provides additional details regarding the type of secondary studies selected for this work, namely the type of work (e.g., conference paper, thesis) and a subtype (e.g., CORE A paper, PhD thesis, Q1 paper) for a finer characterization. To determine the rankings of the conferences and journals, we used the well-known CORE conference ranking and Scimago's Journal Ranking (SJR), respectively.

3 Smart Contract Design Patterns

This section presents our findings with respect to the three research questions identified in Section 2. Full details regarding the results are available at Sadaf et al. (2024). We begin by presenting basic definitions regarding smart contracts, design patterns, and security. We then, in the following subsections, focus on each of the three research questions.

A *smart contract* is a digital agreement with its terms encoded directly into an immutable, decentralized ledger (Khan et al. 2021). The presence of a software fault (i.e., a coding mistake done by a developer) or a vulnerability (i.e., a software fault that can be exploited from the outside to harm the system) in a smart contract has a high impact since the contract itself cannot be corrected; it must be terminated, and a new one must be deployed (Sayeed et al. 2020). Security is generally viewed by the composition of confidentiality, integrity, and availability and is quite an important property in blockchain environments (Zhang et al. 2019). This is due to the usual nature of the applications being deployed, which many times are set in a financial context (i.e., the presence of a vulnerability may lead to very high financial and reputation losses) (Dika and Nowostawski 2018).

Design patterns are solutions for common problems of various kinds. They can be seen as best practice solutions for the identified problems or challenges and help in promoting properties like code reusability or maintainability. They can also be seen as a means to avoid some common mistakes made by developers (i.e., software faults), including vulnerabilities (Wedyan and Abufakher 2020).

Table 2 Characterization of the papers analyzed in this work

Work Type	Count	Subtype	Count
Journal	7	SJR Q1	5
		SJR Q2	2
Conference	35	CORE A	7
		CORE B	3
		CORE C	7
		National	5
		Unranked	13
Workshop	9	–	9
Thesis	7	MSc	5
		PhD	2
			58

In the following subsections, we identify and describe i) the design patterns for smart contracts proposed in the literature, ii) the security design patterns identified during this review, and iii) how well current security design patterns cover known security vulnerabilities.

3.1 Design Patterns for Smart Contracts

We begin by addressing **RQ-1** *"What smart contract design patterns have been proposed in the literature, and what are their goals and characteristics?"*. During this survey, we identified a total of 144 patterns. It is worth noting that, during our analysis of the literature, we encountered certain design patterns, such as "update by delegates" and "one-off access" (Liu et al. 2020b), that were only mentioned by name without any accompanying details or explanations. Consequently, due to the lack of substantive information, we have omitted these patterns from our discussion in the article. The identified design patterns are then placed bottom-up into 28 subcategories, which in turn fit into four general categories according to the objective of the design patterns.

- **Control:** includes design patterns centered on helping manage the control flow, operational login, and functional aspects of smart contracts (Zheng et al. 2020; Wohrer and Zdun 2018).
- **Maintainability:** dedicated to design patterns that aim at reaching longer-lasting contracts and are either centered on facilitating contract upgrades or centered on reducing code complexity (Samreen and Alalfi 2023; Kannengießer et al. 2022).
- **Performance:** design patterns that aid developers in creating smart contracts that are efficient in terms of storage and gas consumption, thus being centered on how storage resources can be effectively used or on how gas is handled by the contract in a way to avoid extraneous costs imposed by the contract execution (Marchesi et al. 2020).
- **Security:** includes design patterns that aim at avoiding the introduction of known vulnerabilities in the code or have a specific security concern as a goal (e.g., authentication, authorization) (Wohrer and Zdun 2018; Dika and Nowostawski 2018).

Figure 2 illustrates the distribution of identified design patterns across the four general categories. As shown, the majority of design patterns fall under the control category (51%), which addresses functional and operational concerns. These concerns are primary in the development of any software system, including smart contracts. The remaining design patterns are roughly divided between security (24%) and performance (20%) patterns, highlighting the equal importance of both aspects in smart contract development. Maintainability (5%) patterns constitute the smallest category, reflecting the less frequent need for maintenance in smart contracts due to the immutability of blockchain technology.

Each category includes several subcategories. Table 3 overviews all categories and the corresponding subcategories identified in this work, along with a conceptual definition of each subcategory.

In the following subsections, we go through the Control, Maintainability, and Performance and corresponding subcategories. Note that the Security category is further detailed in a dedicated section of the paper (i.e., Section 4).

3.2 Control Patterns

Table 4 presents a list of the design patterns found that fit in the Control category.

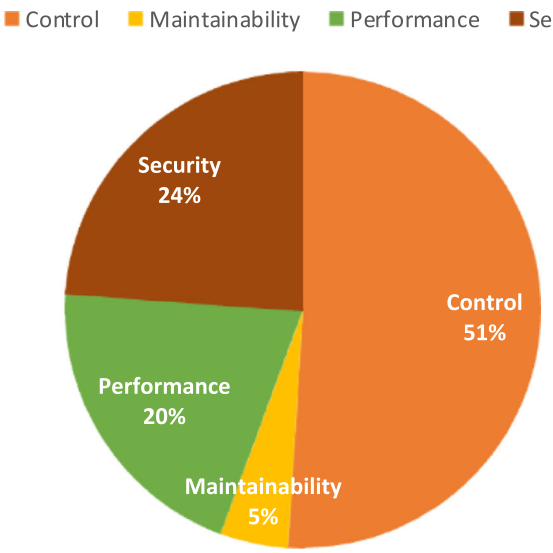

Fig. 2 Distribution of design patterns across the four general categories

Table 3 Design patterns categories and subcategories

Category	Subcategory	Short Description
Control	Data Management	Efficiently managing, storing, and processing data within smart contracts to ensure secure and effective data operations in blockchain systems.
	Wallet and Key Management	Facilitating the creation, storage, and management of cryptographic keys and digital wallets within blockchain applications.
	Business Logic	Enabling the implementation of standardized and modular business logic within smart contracts.
	State Management	Guaranteeing efficient management and consistency of a contract's state within a blockchain framework.
	Fork Management	Developing design patterns focused on identifying blockchain forks and preserving operational stability across divergent chains.
	Life Cycle Management	Defining the different stages of a smart contract's lifecycle, from deployment to termination.
	Registry and Discovery	Supporting the registration and effortless identification of entities within the smart contract and broader blockchain framework.
	Creational	Enabling the instantiation and deployment processes of smart contracts, ensuring efficient setup and initialization in blockchain networks.
	Structural	Dictating the composition and organization of smart contracts and their components to optimize interaction and functionality.
	Interaction	Streamlining and regulating the interactions among smart contracts and between contracts and external entities to ensure efficient perations.

Table 3 continued

Category	Subcategory	Short Description
	Operation Management	Enhancing the supervision and coordination of smart contract functions and processes for improved management.
	Token	Facilitating the creation, issuance, and lifecycle management of digital tokens within blockchain applications.
	Transaction Management	Ensuring efficient execution and coordination of on-chain and off-chain transactions for seamless operations.
Maintainability	Upgrading	Comforting the upgrade process of smart contracts to enhance functionality or address issues.
	Reduce Complexity	Simplifying the architecture of smart contracts to improve their maintainability
	Accessorial Functions	Implementing support functions that aid in the routine maintenance of contracts, such as purging expired records
Performance	Storage Usage	Optimizing for minimal and efficient use of storage resources.
	Gas Consumption	Reducing the amount of gas required for contract execution and transactions.
Security	Secure External Calls	Securing interactions between multiple contracts, protecting against vulnerabilities in contract-to-contract communication.
	Secure Credit Transfer	Enhancing the security of credit transfer mechanisms in financial transactions.
	Secure Programming Practices	Preventing insecure coding practices and compensating for programming language vulnerabilities that could lead to system security issues.
	Flow Control	Managing program execution flow to prevent unauthorized or malicious alterations.
	Safe Arithmetic	Preserving the integrity of arithmetic operations to avert overflow, underflow, and other computational errors.
	Access Control	Securing systems through robust authentication and authorization mechanisms for reliable access management.
	Integrity	Ensuring data remains accurate, consistent, and unaltered throughout its lifecycle, incorporating mechanisms that verify data integrity.
	Privacy	Protecting personal information and ensuring confidentiality across interactions and transactions.
	Risk Mitigation	Implementing strategies to minimize the impact and risk of current security issues.
	Auxiliary	Supplying auxiliary functions or contracts to reinforce the implementation of security solutions.

Table 4 Control Design Patterns

Subcategory	Design Pattern
Data Management	Oracle (Gao et al. 2021; Mühlberger et al. 2020), Key-value Store (Worley and Skjellum 2019), Anti-Oracle (Abreu and Coutinho 2020), Bulletin Board (Worley and Skjellum 2019), Flyweight (Taylor et al. 2022), Content Addressable Storage (Lopez Marin 2022), Immediate-Read (Al-Breiki et al. 2020), Blockchain Data Management(BCDM) (Lamken et al. 2021), Database Connector (Zhang et al. 2020), Raw data on-chain (Xu et al. 2021)
Wallet and Key Management	Master & Sub key creation (Liu et al. 2020a), Shard distribution (Liu et al. 2020b), Shard combination (Liu et al. 2020b), Hot & Cold wallet storage (Liu et al. 2020a)
Business Logic	Poll (Bartoletti and Pompianu 2017), Vote (Worley and Skjellum 2019), Voting (Xu et al. 2021), Lotterie (Bartoletti et al. 2020), Periodic Payment (Bartoletti et al. 2020),
State Management	State Machine (Jurgelaitis et al. 2022), Establish genesis (Xu et al. 2021), State aggregation (Bandara et al. 2020), History oracle (Xu et al. 2021)
Fork Management	Fork check (Bartoletti and Pompianu 2017)
Life Cycle Management	Automatic Deprecation (Wöhrer and Zdun 2018), Availability Checking (Xu et al. 2020)
Registry and Discovery	Contract Registry (Bloo 2018), Multiple registration (Liu et al. 2020b), Entity Registry (Zhang et al. 2020), Identifier Registry (Liu et al. 2020a), Pledge Register (Yang 2021), Consult Directory (Xu et al. 2020), Seller Credential (Lu et al. 2022),
Creational	Contract Factory (Liu et al. 2018), Contract Composer (Moura 2020), Abstract Factory (Taylor et al. 2022), Factory Method (Baryshev 2019), Builder (Baryshev 2019), Singleton (Baryshev 2019), Pledge Template (Yang 2021)
Structural	Contract Facade (Liu et al. 2018), Composite (Baryshev 2019)
Interaction	Contract Mediator (Liu et al. 2018), Contract Observer (Wessling et al. 2019), Address Mapping (Worley and Skjellum 2019), Judge (Worley and Skjellum 2019), Ticker Tape (Worley and Skjellum 2019), Announcement (Worley and Skjellum 2019), Legal and Smart contract Pair (Xu et al. 2018), State Channel (Ordoñez-Guerrero et al. 2022), Dual Resolution (Liu et al. 2020b), Bound with Social Media (Liu et al. 2020a), Request-Response (Al-Breiki et al. 2020), Transfer (Raab 2021), Exchange (Raab 2021), Sharing (Raab 2021), Atomic Swap (Van der Meyden and Maher 2021), User Grants Access (Xu et al. 2020), Asynchronous Request-Response (Xu et al. 2020), Fire-and-Forget (Xu et al. 2020)
Operation Management	Visitor (Baryshev 2019), Role Object (Oruç et al. 2022), Address (Moura 2020), Byteslib (Moura 2020), Context (Moura 2020)
Token	Tokenisation (Wang et al. 2019b), Acquire Token First (Xu et al. 2020), Token Template (Lu et al. 2022), Token Registry (Lu et al. 2022), Policy Contract (Lu et al. 2022), Burned Token (Lu et al. 2022), Token Swap (Lu et al. 2022)
Transaction Management	Subscribe-Notify (Xu et al. 2020), Meta Transactions (Blum et al. 2020), Escrow (Lu et al. 2022), Polling Transaction Pool with Retry (Xu et al. 2020), Polling Blockchain with Retry (Xu et al. 2020)

As we can see in Table 4, the subcategories are relatively diverse. These include classical ones, such as *creational* or *structural* patterns, which can also be found in seminal works, such as Gamma et al. (1993). There are some subcategories that are relatively general and could also apply to other software systems, although at the same time, they carry the specificities of smart contracts, such as the *business logic* or *operation management*. Most of the subcategories truly represent solutions that are blockchain-specific, and this is visible in the names, including *Token*, *Wallet and Key Management*, and also patterns that are more closely related to the processes involving the smart contracts, such as *Life Cycle Management*, *Registry and Discovery*, or *Fork Management*. We must also mention the subcategories holding the highest number of patterns, which are the *Interaction* pattern, which includes solutions for a complex problem in the context of blockchain, namely the fact that contracts interact with each other and also with other external entities, which is also a common source of failures in blockchain applications (Zheng et al. 2020).

3.3 Maintainability Patterns

Table 5 presents the Maintainability category of patterns, which, as we can see, is relatively small. It includes patterns that are centered around code complexity reduction (e.g., to minimize the likelihood of the presence of residual faults in the code), the implementation of functions for routine maintenance of contracts (e.g., purging expired records), or even accounting for possible upgrades of contracts. Although in blockchains like Ethereum, contracts are immutable once deployed (Samreen and Alalfi 2023), there are a few approaches for upgradability, such as using proxy contracts that delegate calls to the actual contracts (Ebrahimi et al. 2024).

3.4 Performance Patterns

Performance patterns revolve around two key aspects in blockchain: storage usage and gas consumption (Marchesi et al. 2020). Most notably, the *Gas Consumption* subcategory gathers various cases of patterns that aim to handle a critical resource in blockchain, which is gas. So, there is a relatively large number of patterns aiming to use gas effectively, such as *Avoid Redundant Operations* or *Limit External Calls*. In the *Storage Usage* category, we highlight the *On-Chain and Off-Chain* pattern, which aims at the frequent need for contracts to store data external to the blockchain. An overview of the identified performance-related patterns, grouped under these two categories, is provided in Table 6.

Table 5 Maintainability design patterns

Subcategory	Design Pattern
Upgrading	Data Segregation (Chatterjee et al. 2020; Wang et al. 2021), Satellite (Bui et al. 2021), Proxy Delegate / Decorator (Antonino et al. 2022)
Reduce Complexity	Use Libraries (Lopez Marin 2022), DynParam (Imeri et al. 2019),
Accessorial Functions	Incentive execution (Rajasekar et al. 2020), Embedded into other functions (Xu et al. 2021)

Table 6 Performance design patterns

Subcategory	Design Pattern
Storage Usage	On-Chain and Off-Chain (Li et al. 2019), Limit Storage (Marchesi et al. 2020), Freeing storage (Marchesi et al. 2020), Tight Variable Packing (Rajasekar et al. 2020)
Gas Consumption	Challenge Response (Lopez Marin 2022), Single Line Swap (Lopez Marin 2022), Use Libraries (Lopez Marin 2022), Delegated Computation (Lopez Marin 2022), Event Log (Marchesi et al. 2020), Limit Storage (Marchesi et al. 2020), Packing Variables (Marchesi et al. 2020), Packing Booleans (Marchesi et al. 2020), Uint* vs Uint256 (Marchesi et al. 2020), Mapping vs Array (Marchesi et al. 2020), Fixed Size (Marchesi et al. 2020), Default Value (Marchesi et al. 2020), Explicitly mark external function (Marchesi et al. 2020), Limit External Calls (Marchesi et al. 2020), Internal Function Calls (Marchesi et al. 2020), Fewer functions (Marchesi et al. 2020), Short Circuit (Marchesi et al. 2020), Short Constant Strings (Marchesi et al. 2020), Limit Modifiers (Marchesi et al. 2020), Avoid redundant operations (Marchesi et al. 2020), Write Values (Marchesi et al. 2020), Freeing storage (Marchesi et al. 2020), Optimizer (Marchesi et al. 2020), Tight Variable Packing (Rajasekar et al. 2020), String equality comparison (Moura 2020), Memory array building (Moura 2020), Gas Consumption Checking (New) (Xu et al. 2020)

3.5 Security Patterns

Table 7 presents the list of Security patterns identified in the literature.

Overall, we identified 36 unique security patterns after removing duplicates. These patterns are categorized into 10 subcategories, with six specifically addressing known security vulnerabilities according to OpenSCV (Lopez Marin 2022). These subcategories are Secure External Calls (e.g., Mutex pattern), Secure Credit Transfer (e.g., Secure Ether Transfer pattern), Secure Programming Practices (e.g., Randomness pattern), Flow Control pattern (e.g., Guard Check), Safe Arithmetic (e.g., SafeMath pattern), and Access Control (e.g., Authorization pattern).

The remaining security patterns do not directly address specific known vulnerabilities but are categorized based on the security measures they implement. These include i) integrity patterns that are designed to ensure the accuracy and consistency of data (e.g., Hash Integrity pattern), ii) privacy patterns that are focused on protecting the confidentiality of information and user identities (e.g., Zero Knowledge Proofs pattern), iii) risk mitigation patterns that help reduce or eliminate the risk of security breaches (e.g., Emergency Stop pattern), and finally, auxiliary patterns that facilitate the implementation of security measures (e.g., Key Generation pattern).

3.6 Summary

Figure 3 presents the distribution of design patterns across different subcategories. As shown in Fig. 3, the Gas Consumption subcategory, within the Performance category, stands out as the largest with 27 design patterns. This significant number highlights the critical importance of gas management in smart contracts. Gas is a fundamental concept in blockchain networks like Ethereum, as it represents the computational effort required to execute operations. Efficient gas management is essential not only for minimizing transaction costs but also for ensuring that smart contracts execute efficiently within the network's resource constraints. The large number of design patterns in this subcategory reflects the ongoing

Table 7 Security design patterns

Subcategory	Design Pattern	Reference
Secure External Calls	Checks Effects Interaction	Wohrer and Zdun (2018); Trojanowska et al. (2020); Britten et al. (2021)
	Mutex	Wohrer and Zdun (2018); Rajasekar et al. (2020); Fahmideh et al. (2022)
	Pull Payment	Wöhrer and Zdun (2018, 2020); Hornyák and Alkhoury (2020)
	Limit External Calls	Marchesi et al. (2020); Moura (2020)
Secure Credit Transfer	Secure ether transfer	Guida and Daniel (2019); Rajasekar et al. (2020); Thirulokachander (2021)
Secure Programming Practices	Randomness	Bartoletti and Pompianu (2017); Chen et al. (2020); Argañaraz et al. (2020)
Flow Control	Guard Check	N'Da et al. (2020); Moura (2020)
	Escrow	Lu et al. (2022)
Safe Arithmetic	SafeMath	Bartoletti and Pompianu (2017); Chen et al. (2020); Moura (2020)
Access Control	Authorization	Bartoletti and Pompianu (2017); Mavridou and Laszka (2018); Argañaraz et al. (2020)
	Time constraint	Bartoletti and Pompianu (2017); Liu et al. (2020b); Xu et al. (2021)
	Commit and Reveal	Wöhrer and Zdun (2018); Liu et al. (2018); Ordoñez-Guerrero et al. (2022)
	Ownership	Verdonck (2019); Töberg et al. (2022); Zecchini et al. (2019)
	Access Restriction	Hornyák and Alkhoury (2020); Wöhrer and Zdun (2018); Moura (2020)
	Blocklist	Worley and Skjellum (2019)
	Dynamic Binding	Lu et al. (2018); Xu et al. (2018); Thirulokachander (2021)
	Embedded Permission	Lu et al. (2018); Rajasekar et al. (2020); Xu et al. (2018)
	Selective Content Generation	Liu et al. (2020b); Xu et al. (2021); Liu et al. (2020a)
	Tokenized Exchange	Zhang et al. (2020)
	Single authorization	Xu et al. (2021)
	Stealth Address	Lu et al. (2022)
	Authorised Spender	Lu et al. (2022)
Integrity	Hash Integrity	Ordoñez-Guerrero et al. (2022); Bandara et al. (2020); Lu et al. (2018)
	File Comparison	Lu et al. (2018)
	Reverse Verifier	Rajasekar et al. (2020); Thirulokachander (2021)
Privacy	Multiple Registrations	Liu et al. (2020b); Xu et al. (2021); Liu et al. (2020a)
	Zero Knowledge Proofs	Blum et al. (2020)

Table 7 continued

Subcategory	Design Pattern	Reference
	Key Sharding	Liu et al. (2020a)
Risk Mitigation	Emergency Stop	Wohrer and Zdun (2018); Guida and Daniel (2019); Ferreira Torres (2022)
	Speed Bump	Wohrer and Zdun (2018); N'Da et al. (2020); Fahmideh et al. (2022)
	Rate Limit	N'Da et al. (2020); Moura (2020); Fahmideh et al. (2022)
	Balance Limit	Hornyák and Alkhoury (2020); Thirulokachander (2021); Rajasekar et al. (2020)
	Multi-signature	Liu et al. (2018); Lu et al. (2018); Bartoletti et al. (2020)
Auxiliary	Key Generation	Lu et al. (2018)
	Database Proxy	Zhang et al. (2020)
	Hash Time Lock Contract	Rajasekar et al. (2020)

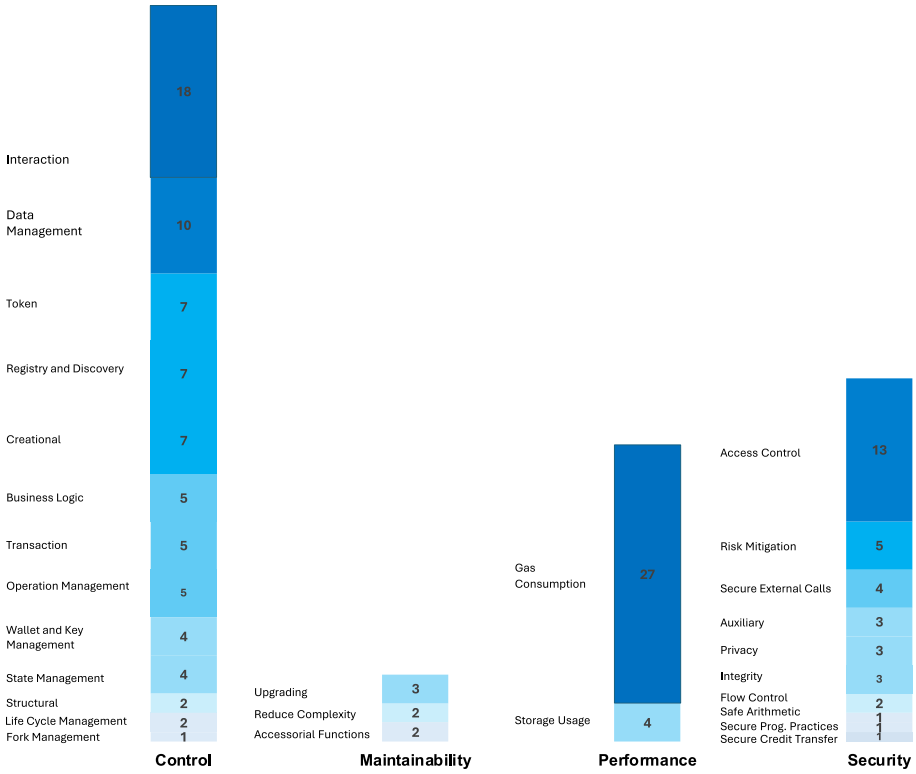

Fig. 3 Number of design patterns in each subcategory

need for developers to optimize smart contracts, reduce unnecessary gas usage, and prevent issues like gas depletion, which can halt contract execution and affect user experience.

The next most substantial subcategory is Interaction, from the Control category. This highlights the complexity and significance of interactions both among smart contracts and among contracts and third-party systems. Interaction patterns are vital because they govern how contracts communicate, share data, and execute interdependent operations. Proper management of these interactions is essential to avoid unexpected behaviors that can arise when multiple contracts or external systems are involved.

The third most significant subcategory is Access Control within the Security category. This emphasizes the crucial need for secure access control in smart contracts and blockchain networks. Access control is a cornerstone of security, ensuring that only authorized users or entities can execute certain functions or access sensitive data within a contract. The significant number of access control patterns reflects the community's recognition of the vital role that rigorous authorization mechanisms play in the security of blockchain applications.

4 Smart Contract Security Design Patterns

This section presents a detailed view of the security design patterns identified in the literature. The goal is to contribute to answering **RQ-2: Which smart contract design patterns target security, and how do they avoid or mitigate security threats?**. It is worth mentioning that the specificities of the blockchain context result in security vulnerabilities (e.g., Reentrancy and Unsafe Non-Blockchain External Call Vidal et al. 2024) and, consequently, security patterns (e.g., Secure ether transfer and Tokenized Exchange) that are distinct from other systems (Vidal et al. 2024). In the following subsections, we go through each subcategory and describe all security patterns in detail.

4.1 Secure External Calls

Secure External Calls is a subcategory of security patterns that includes a set of four patterns found across 15 papers. This subcategory focuses on the security of either inter-contract communication or interaction with entities external to the blockchain. In this subcategory, we find patterns that aim to avoid unsafe external calls (Vidal et al. 2024). The goals are achieved by explicitly checking and ordering calls, concurrency mechanisms like mutexes and decoupling, modifying transfer operations, and limiting external calls as a way of minimizing risks. A well-known issue addressed by these patterns is the reentrancy vulnerability, which occurs when a malicious contract, after initiating a call, is allowed to make new calls to the vulnerable contract before the initial call has been completed. Due to this vulnerability, unexpected state changes may occur, including the depletion of credit (Vidal et al. 2024).

Checks effects interaction This pattern (Rajasekar et al. 2020; Argañaraz et al. 2020) guides smart contract function development and mitigates the risk of introducing reentrancy vulnerabilities by enforcing the order in which operations are performed. A reentrancy vulnerability exists when an external contract maliciously reenters the calling contract (i.e., the contract initially executing) during execution, potentially altering its state or control flow in unintended ways. This pattern specifies that functions should first check all preconditions to validate inputs, ensure authorization, and verify the current state of the contract, protecting against invalid operations. Next, functions should apply effects by securely updating their internal

state, such as adjusting balances or recording changes. Finally, the function should perform interactions with external contracts, such as transferring Ether or invoking external functions, only after ensuring that the contract's state is fully updated and consistent. This order, along with the fulfillment of the preconditions, prevents the external contract from re-entering and altering the state of the calling contract during execution, enforcing the correct flow (Wohrer and Zdun 2018; Britten et al. 2021).

The importance of this pattern is quite evident as the corresponding vulnerability has been exploited in historical incidents, such as the DAO hack, where the lack of proper order enforcement in operations allowed an attacker to exploit the system state through recursive calls, leading to $60 million worth of Ether at the time (Britten et al. 2021). By ensuring that all debits and credits are recorded before any further transactions begin, the Checks Effects Interaction pattern provides a defense against such vulnerabilities (Wohrer and Zdun 2018).

Mutex The main idea behind this pattern is to address the threat of reentrancy attacks by employing a mechanism of mutual exclusion (i.e., a mutex). It ensures that a contract function cannot be re-entered or exploited while still executing by utilizing a boolean variable as a lock. This lock is activated before an external call is made and deactivated once the call is completed, effectively aborting any reentrant calls attempting to execute during the locked state (Wohrer and Zdun 2018). This pattern mirrors the traditional concept of mutexes, where mutual exclusion restricts concurrent access to shared resources, thereby maintaining the integrity of the contract's state. This approach may help in scenarios where recursive calls or callbacks could modify the state of the parent contract before the original function is completed, a vulnerability exploited in reentrancy attacks (William 2008). As a security measure, the Mutex pattern is a straightforward means to protect control flow with minimal negative consequences, such as increased gas costs, complexity, or development overhead. Alternative strategies exist and can also be considered (Rajasekar et al. 2020).

Pull Payment This is an approach to mitigate the risks associated with direct fund transfers in smart contracts, particularly the possibility of failure or vulnerability to reentrancy attacks when a contract sends funds to another party. In traditional "push" payment models, the sending contract initiates a transfer, which involves making an external call to the receiving contract. Such calls inherently hand over control to the receiving contract, potentially allowing malicious reentrant calls that could exploit incomplete state updates or disrupt the contract's execution (Wöhrer and Zdun 2018). To address these challenges, the Pull Payment pattern decouples the transfer action from the contract's core logic by enabling recipients to withdraw funds at their convenience rather than the sender pushing the funds. This approach minimizes the attack surface by isolating each external call and shifting the responsibility for initiating withdrawals to the user, aiming to minimize scenarios where the contract might be forced into unintended states due to failed operations or malicious interactions (Wöhrer and Zdun 2020; Hornyák and Alkhoury 2020).

This pattern may be helpful in scenarios such as micropayments or bidirectional payment channels. For instance, on a blockchain-based crowdfunding platform, contributors' funds could be stored in a contract until the project creator withdraws them. This ensures that if an external interaction fails, the funds remain safely within the contract, preventing loss or unintended behavior.

Limit External Calls Refers to the practice of minimizing the frequency and/or total number of calls to external smart contracts due to inherent costs and associated security risks. Each call to an external contract introduces overhead in the form of gas fees and exposes the system

to potential vulnerabilities, such as reentrancy attacks or unexpected execution failures. This pattern advocates for the use of single, multi-purpose functions that can handle multiple tasks or return various pieces of data in one call, rather than having multiple functions, each performing a separate action or returning a single piece of data (Marchesi et al. 2020; Moura 2020). As an example, a decentralized finance (DeFi) platform might use this pattern to minimize the number of external calls required to execute complex financial transactions, thereby reducing gas costs and potential attack vectors.

In summary, the implementation of the aforementioned set of design patterns improves the security of smart contracts by establishing a framework for secure external calls. These patterns collectively ensure that smart contract functions check preconditions and apply state changes before making external calls, employ mutual exclusion to prevent reentrancy attacks, decouple the action of fund transfers to mitigate risks, and minimize the attack surface by reducing the number and frequency of external calls.

4.2 Secure Credit Transfer

Secure Credit Transfer refers to a subcategory that includes one design pattern discussed in five of the reviewed works. The identified pattern concerns safeguarding transactions involving financial assets. In the context of smart contracts, the transfer of financial assets usually refers to the transfer of cryptocurrencies or tokenized assets between parties. The following pattern was identified with this explicit goal in mind.

Secure ether transfer The Secure Ether Transfer pattern is designed to ensure safe Ether transfer within the Ethereum blockchain, addressing the risks of excessive gas forwarding and reentrancy attacks. It offers three methods for transferring Ether: send, transfer, and call. Send and transfer are higher-level functions with built-in gas limits that protect against malicious contract activity by restricting the amount of gas to be used, whereas call is a low-level function that forwards all remaining gas, ensuring that the gas forwarded does not exceed safe thresholds and preventing reentrancy attacks or other unintended behaviors that could result from forwarding excessive gas to an invoked contract (Rajasekar et al. 2020; Guida and Daniel 2019). As an example, in a smart contract used for crowdfunding, the Secure Ether Transfer pattern ensures that refunds or payouts are executed without exposing the contract to the aforementioned typical risks.

4.3 Secure Programming Practices

In this subcategory, we include patterns that aim at avoiding writing vulnerable code. We identified a single pattern present in six of the analyzed works. This type of pattern may take the form of adhering to a set of coding standards or using specific language mechanisms (e.g., prepared statements for writing SQL). The main idea is that the resultant code either avoids a certain vulnerability (e.g., SQL injection) or is less prone to hold vulnerabilities. Secure programming practices take into account the features of the programming language being used or specific security concerns of the operating environment, such as the Ethereum Virtual Machine (EVM) for Ethereum smart contracts.

Randomness This pattern addresses the challenge of generating random numbers securely within blockchain-based systems that operate in a deterministic environment, i.e., where the output of a function must be the same for all nodes given the same input to maintain consensus

(Bartoletti and Pompianu 2017; Argañaraz et al. 2020). This deterministic nature poses obstacles to achieving true randomness, as predictable or insufficiently random values can lead to security vulnerabilities that can be exploited to manipulate outcomes in applications such as lotteries, games, and auctions (Lopez Marin 2022; Chen et al. 2020).

To mitigate the insufficient randomness risks, the Randomness pattern often relies on oracles, which provide random values generated off-chain or on-chain techniques like leveraging block hashes from not-yet-mined blocks. For example, a lottery smart contract might query an off-chain oracle service to ensure secure and unbiased randomness. Although on-chain methods, such as using future block hashes, are more convenient, they are generally considered less secure due to potential manipulation by miners or attackers with substantial computational power (Chen et al. 2020). The implementation of the Randomness pattern ensures that the generation of random numbers is reusable and modular (Lopez Marin 2022).

4.4 Flow Control

Design patterns that are categorized under Flow Control are found across three of the analyzed papers. These patterns aim to prevent an adversary from being able to manipulate the execution flow of the program and, consequently, the execution order of statements or state transitions within a program. The goal is to ensure that the program behaves as intended, even in the presence of unexpected or malicious inputs that could try to alter the program's flow to exploit vulnerabilities. The following paragraphs present the patterns that fit in this subcategory.

Guard Check Regulates control flow to enhance security by ensuring that all predefined conditions on the contract's state and function inputs are verified before execution proceeds so that expected behavior is assured. In practice, this allows the blocking of unauthorized actions or reduces the likelihood of activating residual faults in the code with consequences in the control flow of the code (N'Da et al. 2020; Moura 2020).

Guard checks can include mechanisms such as user authentication, validation of input data, or compliance with conditions that must hold for the function to execute correctly (N'Da et al. 2020; Moura 2020). In practice, they can validate user inputs, verify the contract's state, enforce invariants, and ensure compliance with logical constraints. They may also discard unacceptable conditions, thereby avoiding vulnerabilities or unintended outcomes. For example, these checks could prevent an unauthorized user from accessing privileged functions or stop a transaction if the contract state does not meet predefined criteria. The utility of this pattern extends to various scenarios, such as user authentication, data validation, or verifying the integrity of responses from other contracts (Volland 2019).

Escrow The Escrow design pattern is focused on transaction management by mediating the transfer of tokens between two parties through a neutral third-party contract called the escrow. In this pattern, tokens are first transferred to the escrow smart contract, which serves as a trusted intermediary to hold the funds until predetermined payment conditions are met (e.g., coupons, insurance payouts, social welfare benefits) (Lu et al. 2022). This mechanism ensures that the seller delivers the promised product or service and that the buyer completes the payment accordingly, mitigating the risk of failure to comply with the agreement. The escrow contract's logic includes verification steps to confirm that all transaction requirements have been fulfilled before releasing the funds to the seller. This pattern is helpful, especially in high-value transactions where the impact of a failure or malicious behavior is high and,

as a consequence, the need to protect the interests of all parties is strong (Lu et al. 2022). This pattern is helpful, especially in high-value transactions where the impact of a failure or malicious behavior is high and, as a consequence, the need to protect the interests of all parties is strong, such as insurance payouts, social welfare benefits, or e-commerce settlements (Goldfeder et al. 2017; Amani et al. 2018).

4.5 Safe Arithmetic

Safe Arithmetic patterns, found across four papers, refer to techniques that ensure arithmetic operations are carried out with correctness and, as a consequence, do not compromise the intended functionality (Hickey et al. 2001). In the particular case of smart contracts, there are numerous cases of calculations resulting in either a number larger than the maximum value the data type can hold (i.e., overflows) or a number that is smaller than the smallest value the data type can represent (i.e., underflows) (Sun et al. 2022). Such cases have been many times reported as security vulnerabilities, allowing an attacker to manipulate balances or other important state variables (Sayeed et al. 2020; Chen et al. 2020). Safe arithmetic is essential to avoid this kind of issue.

SafeMath This pattern, which is available in a few libraries (Chen et al. 2020), allows avoiding overflow or underflow vulnerabilities. Overflow occurs when an arithmetic operation exceeds the maximum value of a data type, while underflow arises when an operation results in a value below its minimum range (Vidal et al. 2024). Such vulnerabilities have historically led to severe exploits, as demonstrated in the attack on BEC tokens, where improper validation allowed unauthorized manipulation of balances and state variables (Tan et al. 2022). The main idea is to provide safe versions of the four arithmetic operations by including checks against problematic outcomes. Whenever a calculation reaches an overflow or underflow state, SafeMath reverts the transaction to ensure the integrity of the state of the contract (Moura 2020; Bartoletti and Pompianu 2017). While this approach incurs some runtime overhead and gas costs, the benefits of preventing critical arithmetic vulnerabilities outweigh the costs (Ren et al. 2021; Bartoletti and Pompianu 2017).

4.6 Access Control

The Access Control subcategory includes a set of patterns, found across 30 of the analyzed papers, that mostly target authentication and authorization, which are respectively related to identity verification and checking if a certain entity is allowed to execute a certain operation (Samarati and De Vimercati 2000; Penelova 2021).

Authorization This pattern restricts the execution of certain functions that are sensitive or somehow critical (e.g., sending ether, invoking the *self-destruct()* function (Mavridou and Laszka 2018)) to specific user addresses. It is commonly used to ensure that only authorized users, such as the contract owner or those with specific roles, can invoke certain functions that may alter the contract's state. In practical terms, implementing this pattern typically involves using access control lists or mappings of addresses to track permissions and incorporating modifiers in the contract's functions to check these permissions before allowing the execution of the code (Bartoletti and Pompianu 2017; Argañaraz et al. 2020).

Real-world applications of the Authorization pattern are obviously widespread. For instance, in notary contracts, the pattern may ensure that only the document owner can add or modify data, thereby preventing external tampering. In gambling games, the pattern

may allow only the contract owner to manage participants' fees and rewards, for instance, ensuring that only the winner can withdraw their prize. Some other decentralized applications may also generally use this pattern to assign different roles to different user addresses (Bartoletti and Pompianu 2017).

Time Constraint This pattern is used to define the time window during which certain actions are allowed or to specify deadlines after which certain functions can no longer be used. In practice, the developer controls the execution of contract functions by using timestamps or block numbers as references for measuring time intervals and durations, thus adding temporal logic to contract operations. These mechanisms will ensure that contract actions are bound by the temporal rules encoded within the contract (Bartoletti and Pompianu 2017; Xu et al. 2021). This pattern may be particularly useful in scenarios where the timing of actions is critical, such as financial transactions, identity verification, and gaming events (Bartoletti and Pompianu 2017). For example, a certificate holder might share a link to a verifier that grants access to credentials only within a defined period, preventing access after the period has expired (Xu et al. 2021; Hornyák and Alkhoury 2020).

Commit and Reveal This pattern consists of two phases: a commit phase and a reveal phase. In the Commit Phase, a user submits a hashed version of its information to the blockchain, keeping the original information secret for a period of time. In the Reveal Phase, the user discloses the original information, and its integrity can be verified by the network by hashing it again and comparing it to the initially committed hash (Wöhrer and Zdun 2018; Liu et al. 2018).

The commit-reveal scheme is crucial in some blockchain-based applications because it ensures that no one can access the information before its original content is revealed, and it prevents users from altering their submissions after committing them (Rajasekar et al. 2020). For example, in a blockchain-based auction, bidders can commit their bids without revealing the corresponding details to other participants until the auction ends. Once it ends, the details will be verifiable. Thus, this pattern actually addresses both access control and integrity.

Ownership This is an access control pattern used in smart contract development to ensure that critical functions are only callable by a specific, authorized entity - the owner of the contract. The owner is set at the moment the contract is deployed, with the deployer's address being then registered as the owner and this address being used to verify authorization for executing sensitive methods (Töberg et al. 2022; Zecchini et al. 2019; Six et al. 2022a). For example, maintenance tasks or functions that manage contract funds are typically restricted to the owner to prevent potential misuse (Wöhrer and Zdun 2018, 2020). Real-world examples of the Ownership pattern include its use in decentralized finance platforms, where the contract owner might need to update interest rates or manage liquidity pools.

Access Restriction The Access Restriction pattern ensures that certain functions can only be executed when specific restrictive conditions are met. The pattern aims to control function access based on predefined requirements, which can include temporal conditions, caller information, transaction details, or other necessary prerequisites. These requirements are often implemented using reusable modifiers that enforce conditions across multiple functions within the same contract. Additionally, Access Restriction can extend permissions to a broader set of users, such as those approved by other members, thus providing more flexible access control (Wöhrer and Zdun 2018; Hornyák and Alkhoury 2020). For example, in a decentralized finance application, an Access Restriction pattern might be used to ensure

that only users who have staked a certain amount of tokens can access premium features or execute high-value transactions (Wöhrer and Zdun 2018; Hornyák and Alkhoury 2020).

Blocklist The Blocklist pattern addresses the need to restrict access to certain entities regarding an off-chain asset. This is achieved by implementing a contract that maintains a set of Bloom filter coefficients. These coefficients enable efficient off-chain computation, allowing clients to determine if an entity is included in the blocklist without explicitly storing the list within the contract. The main components of this pattern include a contract that holds the Bloom filter coefficients and the off-chain clients that utilize these coefficients to perform the necessary calculations, ensuring that disallowed entities are blocked from accessing the specified asset (Worley and Skjellum 2019). This pattern is useful in scenarios where maintaining a comprehensive list of disallowed entities on-chain would be impractical due to storage constraints (Wei et al. 2022). For example, a contract could use this pattern to track certificate revocations or the Ethereum addresses of stolen cryptocurrency, ensuring that these entities are blocked from accessing the specified asset.

Dynamic Binding The goal of this pattern is to authorize actions by participants whose addresses are not known in advance. The pattern works through the association of an authority with a contract action through an off-chain secret rather than predefining the user's address within the contract. The secret, when provided by the user, is verified by comparing its hash with a hash stored in the smart contract. This approach allows the smart contract to be updated to include this verification mechanism without needing a special protocol to exchange the secret. A third-party contract can also facilitate the authorization by confirming the secret key and then allowing interaction with the target contract (Thirulokachander 2021).

Embedded Permission This pattern enforces access restrictions directly within the contract's code for selected functions. By specifying authorized addresses within the contract, this pattern allows only those pre-approved participants to invoke critical functions, such as *self-destruct()* in Solidity. It embeds a check within the function itself that verifies whether the caller's address matches the list of authorized addresses before any function logic is executed (Lu et al. 2018; Rajasekar et al. 2020). The presence of embedded permissions could help in avoid the Parity multi-sig wallet incident, which ended up in freezing 500K Ether (Xu et al. 2018; Rajasekar et al. 2020).

Selective Content Generation This pattern enables the creation of customized credentials tailored to a holder's specific requirements regarding the credential contents. In this pattern, an issuer generates a credential that includes only the necessary data as requested by the holder, ensuring that only the essential information is shared with a verifier, who needs this information to validate the holder's identity (Liu et al. 2020b). This approach allows data owners to specify access control rules, thereby generating responses that avoid leaking unnecessary data (Xu et al. 2021; Liu et al. 2020a). As an example, this pattern can be applied in contexts such as Self-Sovereign Identity (SSI), where a certificate issuer might generate a customized credential to show only the grade earned for a module rather than exposing the entire transcript (Xu et al. 2021; Liu et al. 2020a).

Tokenized Exchange This pattern allows authorizing and tracking access to off-chain data storage through the use of security tokens. It allows for a verifiable history of data access, which is useful for auditability, and also encapsulates the complexities of interacting with various off-chain data sources via a standardized interface that allows encoding a set of

attributes describing the sources and basic operations that can be performed on them Zhang et al. (2020). A real-world application of this pattern could be in healthcare, where patient data stored off-chain needs to be accessed by various stakeholders such as doctors, insurance companies, and researchers.

Single Authorization The Single Authorization design pattern in smart contract security is a protocol that designates a specific blockchain account as the sole authority to sign and authorize transactions (Xu et al. 2021). This pattern enforces function-level authorization to mitigate the risks associated with the open accessibility of smart contracts on blockchain platforms, where, typically, any participant could invoke any function. It basically requires that each function invocation is preceded by a verification step to confirm the caller's authorization by the predefined single authority, offering a centralized method to manage permissions in contrast to more complex multi-signature authorization mechanisms. For example, in a decentralized finance application, this pattern could be used to ensure that only a designated account can authorize high-value transactions, thereby protecting the contract from unauthorized access and other potential risks (Xu et al. 2021).

Stealth Address This pattern ensures that only the designated recipient can detect and spend the funds associated with a specific transaction in the blockchain. When a sender initiates a transfer, it generates a one-time, computationally random-looking address known as a stealth address. This address is unique to the transaction and can only be recognized by the recipient, who can then claim and spend the funds. This pattern prevents any third party from determining the actual recipient of the transaction. Stealth addresses obscure the economic relationship between the sender and recipient while ensuring that only the intended recipient has access to the transferred assets (Lu et al. 2022; Kovács and Seres 2024).

Authorized Spender The Authorized Spender design pattern in smart contract development enables token owners to grant a third party, such as another user or a smart contract, permission to spend a predefined amount of their tokens. The pattern is implemented through an 'approve' function within the token contract, where the token owner specifies the spender's address and the token amount they are authorized to use. By doing so, the contract records an allowance that is verified each time the authorized spender attempts to initiate a transfer, ensuring that the spender cannot exceed the set limit and providing a secure method for delegated transactions. This is particularly useful in blockchain ecosystems where buyers want to delegate payment responsibilities (Lu et al. 2022). As an example, in a decentralized marketplace, a buyer can authorize a smart contract to handle payments on their behalf, streamlining the transaction process and enhancing user convenience. The approve function can be added to the token template contract, with parameters for the spender's address and the amount, allowing, in this way, the occurrence of delegated transactions.

4.7 Integrity

The Integrity subcategory of the security category, which includes a set of three patterns found across 11 papers, ensures data is kept from unauthorized changes throughout its lifecycle. It includes mechanisms that verify data integrity, protect against unauthorized modifications, and ensure the authenticity of both on-chain and off-chain information.

Hash Integrity This pattern aims to inform about the integrity of data that is stored outside the blockchain. A hash of the original data is generated and stored on the blockchain, which allows users to check the integrity of the off-chain data by checking the on-chain hash value (Lu et al. 2018; Ordoñez-Guerrero et al. 2022). This pattern is essential for the case of applications in which data integrity is critical, such as financial records, legal documents, and other integrity-sensitive information (Xu et al. 2021; Rajasekar et al. 2020).

File Comparison File comparison involves verifying the integrity of a file by comparing its cryptographic hash value with the corresponding hash stored on the blockchain. This service allows users to check if a file has been altered without authorization, ensuring that the file remains unchanged and original. By utilizing a cryptographic hash value like SHA-256, users can select a file and verify its authenticity. If the hashes match, the file is confirmed to be authentic. This approach emphasizes verification at the file level rather than handling extensive data that may not be easily stored on the blockchain (Lu et al. 2018; Parmar and Kaur 2021). As an example, in auditing processes, the hash of an agreement stored on-chain allows auditors to verify the integrity of an off-chain file by ensuring its hash matches the blockchain-stored value (Lu et al. 2018).

Reverse Verifier The Reverse Verifier pattern ensures the integrity of blockchain data when accessed by external sources. When an off-chain entity, like a traditional database application or a banking API, requests blockchain data, this pattern verifies it by checking the stored hash against the data, detecting any tampering. Reverse Verifier specifically ensures the integrity of blockchain data as it is accessed by external systems. This is especially useful for scenarios like retrieving check data from the blockchain (Rajasekar et al. 2020).

4.8 Privacy

This subcategory includes three design patterns found across four of the analyzed papers. These patterns focus on ensuring confidentiality across interactions and transactions, e.g., by hiding identities or preventing identity correlation.

Multiple Registrations The Multiple Registration pattern allows an entity to register and use distinct identifiers for each of its transacting relationships, minimizing the risk of identity correlation and privacy breaches. By using separate identifiers for different applications or roles, interactions with one party can be isolated from another, thus enhancing privacy. This approach involves registering multiple identifiers in an identifier registry, where each identifier uniquely corresponds to a different identity relationship. Consequently, each transaction is associated with a unique identifier, preventing the linkage of identities and protecting the entity's privacy by avoiding the exposure of all identities held by that entity through a single identifier (Xu et al. 2021; Liu et al. 2020a). This approach leverages decentralized identifiers, a widely adopted standard in Self-Sovereign Identity frameworks, to manage unique, persistent identifiers associated with blockchain accounts (Liu et al. 2020b; Xu et al. 2021). As an example, a person could register separate identifiers for interactions with a hospital and a university, preventing either entity from linking his/her activities (Xu et al. 2021; Liu et al. 2020a).

Zero Knowledge Proofs Zero-knowledge proof allows users to prove the validity of transactions or data without revealing the underlying details. This pattern allows one party (the

prover) to inform another party (the verifier) that they possess certain secret information, which is verifiable, without revealing the information itself or the identities of either party. This is achieved through an interactive verification protocol consisting of a sequence of predefined actions that ensure the verifier is convinced of the prover's knowledge without gaining any additional data. The protocol does not require complex public key infrastructure and is resistant to malicious attempts to extract further information (Blum et al. 2020; Sun et al. 2021; Wang et al. 2019a). A practical application may be the encoding of a vulnerability and corresponding exploit of a real system. This would be verifiable by a target entity but, at the same time, would not reveal the details, which otherwise could allow other entities to exploit the vulnerable system.

Key Sharding In this pattern, a master cryptographic key is split into multiple shards, which alone are incomplete but collectively can be used to reconstruct the original key. This technique prevents a single key's overexposure across multiple transactions, protecting identity privacy. It enables an entity to manage distinct subkeys for various identity accounts under one master key, mitigating risks associated with key compromise. The main idea is that entities can conduct transactions with the assurance that their various identities remain unlinked and secure (Liu et al. 2020a; Wang et al. 2019a). In practical applications, users can store these key shards securely in diverse ways, such as distributing them among trusted family members or storing them in physical safes. If the original key is lost-due to events like device failure or theft-users can recover it by collecting the required number of shards that meet the regrouping threshold (Huang et al. 2024; Liu et al. 2020a).

4.9 Risk Mitigation

This subcategory includes five design patterns found across 16 of the analyzed papers. These patterns aim to reduce the impact of possible existing security threats in smart contracts.

Emergency Stop This pattern is a measure to rapidly stop the impact of a major vulnerability by allowing to securely stop contract execution (Thirulokachander 2021). In practice, it can be implemented as a function within the contract that an authenticated party can activate so that the problematic operations can be disabled (Wohrer and Zdun 2018). A real-world example of the need for this pattern is illustrated by the DAO attack, where participants on the Ethereum network could only watch as their funds were withdrawn due to a lack of an emergency stop mechanism (Rajasekar et al. 2020).

Speed Bump This pattern delays the execution of sensitive operations to diminish the risk of abrupt, potentially damaging activities (e.g., an attacker exploiting a vulnerability). The implementation of this pattern may take several forms, such as limiting the amount of funds a user can withdraw per day and instituting a mandatory waiting period before processing withdrawals, among others. This provides additional time for stakeholders to detect and respond to irregularities (Wohrer and Zdun 2018; Fahmideh et al. 2022). Analogous to real-world banking scenarios, such as limiting withdrawals during a bank run, this pattern prevents sudden, unrestrained actions that could harm the integrity of a contract. For example, in decentralized finance (DeFi), slowing down high-value transactions allows additional time for validators or stakeholders to monitor irregularities (Thirulokachander 2021).

Rate Limit Is a pattern that allows contracts to manage and cap the frequency of task executions over a specified time period, with the aim of mitigating the effect of a high number of messages being sent to a smart contract (e.g., a surge of requests) and controlling its computational demand (Wohrer and Zdun 2018). By setting a rate limit, the pattern ensures that functions, such as withdrawals or other sensitive operations, are not called too frequently, thereby avoiding the risk associated with scenarios like rapid fund drainage (Fahmideh et al. 2022; Moura 2020).

Balance Limit This design pattern refers to setting the maximum amount of funds that can be held within a contract at any given time, thereby managing the amount of money at risk. When the contract's balance reaches this predefined quota, any further payments that attempt to exceed this limit are rejected. However, it is important to note that this approach cannot prevent Ether from being forcibly sent to the contract. This limits the potential damage in the event of a contract breach by ensuring that only a manageable amount of funds is ever at risk (Wohrer and Zdun 2018; Thirulokachander 2021). By capping the funds at risk, this pattern minimizes the attack surface, as contracts holding excessive capital become attractive targets for malicious actors (Wohrer and Zdun 2018).

Multi-signature The goal of this pattern is to make a certain transaction require the approval of multiple authorized parties before it can be validated and executed. The main idea is that no single entity should be able to unilaterally carry out a transaction. It serves as an additional protective layer, particularly in scenarios where the loss of a private key cannot be remedied by current blockchain technology. Multi-signature setups can vary in configuration, from requiring all parties to sign a transaction to requiring a subset of signatures. This allows for some flexibility in the mechanism, e.g., to better fit specific scenarios (Liu et al. 2018; Lu et al. 2018).

4.10 Auxiliary

The Auxiliary category includes a set of patterns found across three papers. They aim to facilitate the implementation of some security-related specific function within a system, thus not being part of the core functionality but instead supporting the security measures.

Key Generation This pattern refers to the creation and distribution of encryption keys to secure transactions on the blockchain. The overall idea is to generate keys (using asymmetric encryption) that are shared among parties prior to executing transactions on the blockchain. Such keys should be properly managed so that security is not compromised (Lu et al. 2018).

Database Proxy The Database Proxy pattern is a specialized adaptation of the traditional Proxy design pattern tailored for blockchain-based systems. Its primary purpose is to add an extra layer of security and efficiency by performing lightweight tasks, such as basic security checks or auditing, before allowing access to database connectors. It minimizes computational costs on-chain by deferring the retrieval and unpackaging or decryption of sensitive data until it is required off-chain. The contract code should be organized to first invoke the proxy for preliminary operations and then proceed with more detailed data handling off-chain (Zhang et al. 2020).

Hashed Timelock Contract (HTLC) The Hashed Timelock Contract (HTLC) pattern relies on hashlocks and timelocks to ensure secure and atomic swaps by requiring recipients to provide cryptographic proof of payment within a specified timeframe. Technically, the HTLC pattern enables the implementation of time-bound transactions (Bartoletti et al. 2020; Monika et al. 2022; Chan and Lesani 2021).

4.11 Summary

Within the Security category, we have identified 10 distinct subcategories, each representing a critical aspect of securing smart contracts and blockchain applications. Among these, the Access Control subcategory stands out as the most significant, comprising 13 design patterns. This is reflective of the crucial role that access control plays in ensuring that only authorized entities can execute specific functions or access sensitive data within a smart contract. The large number of design patterns in this subcategory also highlights the diverse approaches available to developers for implementing strong access control, from simple ownership checks to more complex permission management systems.

Following Access Control, the Risk Mitigation and Secure External Calls subcategories are the next largest, respectively, with 5 and 4 patterns.

Risk Mitigation includes design patterns aimed at reducing the potential risk of security threats and vulnerabilities. These patterns help developers anticipate and address potential risks that could compromise the contract's security and functionality. The presence of 5 patterns in this subcategory highlights the importance of proactively managing risks in smart contract development.

Secure External Calls includes design patterns focused on safely interacting with external contracts or systems. Smart contracts often need to communicate with other contracts or external data sources, which introduces a significant security risk if not handled properly. Vulnerabilities such as reentrancy attacks can arise when external calls are not securely managed. The four design patterns in this subcategory provide developers with some best practices for securely handling these interactions.

These subcategories, together with others in the security category, form a more complete framework for addressing the various security challenges inherent in smart contract development. The emphasis on Access Control, Risk Mitigation, and Secure External Calls reflects the broader priorities in blockchain security: mitigating security threats, controlling access, and ensuring safe interactions with external entities.

5 Mapping Between Security Design Patterns and Vulnerabilities

This section aims to answer **"RQ-3 Which smart contracts' vulnerabilities are directly addressed by the identified security design patterns?"**. In this section, we aim to provide a more in-depth evaluation of security design patterns, identifying which vulnerabilities are effectively mitigated by existing patterns and highlighting any gaps where current design patterns may not adequately address smart contract security vulnerabilities.

To map the design patterns to vulnerabilities, we rely on the OpenSCV vulnerability classification (Vidal et al. 2024), which, at the time of writing at to the best of our knowledge, is the most comprehensive smart contract taxonomy available. It covers 94 known smart contract vulnerabilities, categorized into 8 categories: i) Unsafe External Calls, ii) Mishandled Events, iii) Gas Depletion, iv) Erroneous Credit Transfer, v) Bad Programming Practices and

Language Weaknesses, vi) Incorrect Control Flow, vii) Arithmetic Issues, and viii) Improper Access Control. Each category is decomposed into several subcategories.

To map patterns and vulnerabilities, we examined the intended purpose of each design pattern and navigated through the OpenSCV tree to find a possible target vulnerability. This was done in two steps. First, we followed through the hierarchical structure, which should allow us to quickly navigate to a given vulnerability or a set of vulnerabilities. If no match was found, we then conducted an exhaustive search through all vulnerabilities to identify a target based on the intended purpose of the design pattern. This process involved an initial iteration by an early-stage researcher, followed by verification from an experienced researcher.

Among the security design patterns identified in the previous section, we found 5 patterns (i.e., Checks-Effects-Interaction, Mutex, Randomness, SafeMath, and Ownership) that specifically target one or more known smart contract vulnerabilities. These patterns collectively address a total of 6 specific vulnerabilities. In total, 13 design patterns are related to 5 subcategories of vulnerabilities, and 22 patterns are related to 6 categories of vulnerabilities without addressing any specific known vulnerabilities. The following paragraphs detail the connections between the design patterns and OpenSCV vulnerabilities.

Checks Effects Interaction and **Mutex** address two vulnerabilities from *1 Unsafe External Call*[1] category and *1.1 Reentrancy* subcategory, namely *1.1.1 Unsafe Credit Transfer* and *1.1.2 Unsafe System State Changes*.

The Unsafe Credit Transfer vulnerability enables attackers to manipulate balances by initiating withdrawal operations before the balance is updated. This allows a malicious contract to withdraw funds multiple times, exceeding its actual balance. The Unsafe System State Changes vulnerability is similar to the Unsafe Credit Transfer vulnerability but does not involve user funds. Instead, it allows an attacker to disrupt the contract's state by initiating a call before a previous one has been completed, potentially leading to unexpected states.

Checks Effects Interaction addresses these vulnerabilities by enforcing the order in which operations are performed, and Mutex addresses these vulnerabilities by ensuring that a contract function cannot be re-entered and potentially exploited while it is still being executed.

The **Randomness** addresses *5.1 Bad Randomness* vulnerability from *5 Bad Programming Practices and Language Weaknesses* category. *5.1 Bad Randomness* refers to the generation of random values that are not truly random or can be predicted by an attacker. Randomness pattern provides a way to generate a random number in a deterministic blockchain system.

The **SafeMath** addresses both *7.1.1 Integer Underflow* and *7.1.2 Integer Overflow* from the *7.1 Overflow and Underflow* subcategory and *7 Arithmetic Issues* category by providing safe versions of the four arithmetic operations, incorporating checks to prevent problematic outcomes.

The **Ownership** pattern addresses *8.1.2 Owner Manipulation* from the *8.1 Incorrect Authentication or Authorization* and *8 Improper Access Control* category of vulnerabilities. The Owner Manipulation vulnerability allows an attacker to exploit a function or feature of the smart contract by manipulating the owner control variable, enabling the attacker to perform restricted operations. The Ownership pattern addresses this issue by ensuring that critical functions can only be executed by a specific, authorized entity-the contract owner.

The remaining design patterns do not address specific vulnerabilities but are associated with broader subcategories and categories of vulnerabilities.

The **Authorization**, **Access Restriction**, **Embedded Permission**, **Single Authorization**, **Stealth Address**, and **Authorized Spender** patterns address the *8.1 Incorrect Authentication*

[1] We maintain the numbering of vulnerabilities according to the OpenSCV classification to facilitate easier identification and tracking.

or Authorization subcategory within the *8 Improper Access Control* category of vulnerabilities. These vulnerabilities typically arise when smart contracts fail to properly identify a client or determine their privileges. Authorization mitigates this risk by restricting specific functions (such as sensitive or critical ones) to designated user addresses. Access Restriction ensures that certain functions can only be executed under specific restrictive conditions. Embedded Permission includes a check within the function to verify that the caller's address is on an authorized list before executing the function logic. Single Authorization enforces function-level access control by requiring each function call to be verified by a predefined authority, simplifying permission management compared to multi-signature methods. Stealth Address ensures that only the intended recipient can detect and use the funds associated with a specific transaction. Authorized Spender allows token owners to grant a third party, such as another user or a smart contract, permission to spend a specified amount of their tokens.

Commit and Reveal and **Selective Content Generation** address the *8.2 Improper Protection of Sensitive Data* subcategory within *8 Improper Access Control*. Commit and Reveal allows users to conceal the original content of their sensitive information for a period of time while also enabling the verification of the information's integrity once it is disclosed. Selective Content Generation facilitates the creation of customized credentials tailored to the holder's specific needs. The issuer generates a credential containing only the necessary data requested by the holder, ensuring that only essential information is shared with a verifier, who uses it to validate the holder's identity.

Time Constraint , **Blocklist**, **Dynamic Binding**, and **Tokenized Exchange** are related to the *8 Improper Access Control* category. The Time Constraint pattern restricts access to functions by setting deadlines after which the functions are no longer available. Blocklist controls access to off-chain assets by maintaining a list of entities that are denied access. Dynamic Binding enables authorization for actions by participants whose addresses are not known in advance. Tokenized Exchange manages and tracks access to off-chain data storage through the use of security tokens.

The **Pull Payment** and **Limit External Calls** are generally related to the vulnerabilities within *1 Unsafe External Calls* category. Pull Payment tries to reduce the risk of external calls by allowing the recipient to withdraw funds instead of the sender pushing them, and Limit External Calls does it by minimizing the frequency and/or total number of calls to external smart contracts.

Secure ether transfer in general is related to *4 Erroneous Credit Transfer* category of vulnerabilities. It aims to promote secure credit transfers by offering three methods (i.e., send, transfer, and call) that either have built-in gas limits or forward all remaining gas to the invoked contract, which may indirectly prevent unintended behaviors like reentrancy (Rajasekar et al. 2020; Guida and Daniel 2019).

Guard Check and **Escrow** are both associated with the *6 Incorrect Control Flow* category of vulnerabilities. Guard Check ensures the correct flow of transactions by verifying that all predefined conditions on the contract's state and function inputs are met before execution proceeds. Escrow uses a mediator to facilitate token transfers, ensuring that payment conditions are met before the seller delivers the product or service and the buyer completes the payment.

The remaining design patterns do not directly address specific known vulnerabilities in smart contracts or their associated categories and subcategories. They instead focus on

enhancing security by facilitating the implementation of security measures (e.g., Key Generation, Database Proxy, and Hashed Timelock Contract (HTLC) facilitate the implementation of security measures) and properties such as integrity (e.g., Hash Integrity, File Comparison, and Reverse Verifier help in insurance of data integrity) and privacy (e.g., Multiple registrations, Zero Knowledge Proofs, and Key Sharding help to hide identities) or by mitigating the risks of security attacks or failures (e.g., Emergency Stop, Speed Bump, Rate Limit, Balance Limit, Multi-signature help in mitigating the risk of security problems).

Table 8 presents a summary of the mapping between the design patterns and the security vulnerabilities.

6 Discussion

We have identified 144 smart contract design patterns and categorized them into four main groups: Control, Maintainability, Performance, and Security. This categorization provides a structured framework for understanding various critical aspects of smart contract development. We then highlighted several design patterns, with a particular emphasis on security.

Here, we highlight the key findings and gaps identified from our analysis:

- **General distribution of design patterns across the main categories:** The majority of design patterns fall under the Control category (51%), addressing functional and operational concerns that are fundamental to software development, including smart contract development. The remaining design patterns are nearly evenly distributed between Security (24%) and Performance (20%), underscoring the importance of both security and performance aspects in smart contract development. Patterns related to Maintainability make up the smallest category (5%), reflecting the reduced need for maintenance in smart contracts due to the immutable nature of blockchain technology.
- **Distribution of design patterns across the subcategories:** The gas consumption subcategory within the performance category is the largest, comprising 27 design patterns. This underscores the critical importance of gas management in blockchain and smart contracts. The next most significant subcategory is interaction, within the control category, highlighting the complexity of developing interactions among contracts and among contracts and third parties. The third most significant subcategory is access control within the security category, emphasizing the crucial need for secure access to smart contracts and blockchain. Together, these subcategories, Gas Consumption, Interaction, and Access Control, highlight the need for careful attention to performance optimization, interaction management, and security controls for smart contract development, each of which is crucial for building reliable, efficient, and secure blockchain applications.
- **Categorization of security design patterns:** We have identified a total of 36 security design patterns, categorized into 10 subcategories. For each security pattern, we discussed the general concept, the intention behind it, and the security measures it enables. Whenever possible, we connected the descriptions to real-world examples or provided a more practical perspective on each pattern.
- **Distribution of security design patterns across security subcategories:** Within the Security category, we identified 10 key subcategories critical to securing smart contracts and blockchain applications. Access Control is the most significant, with 13 design patterns, highlighting its important role in ensuring that only authorized entities can access or modify smart contract functions and data. Following this, Risk Mitigation and Secure External Calls are the next largest subcategories, respectively, with 5 and 4 patterns. Risk

Table 8 Mapping between security design patterns and vulnerabilities

Subcategory DPs	Security DPs	Category of Vulnerability	Vulnerability Subcategory	Vulnerability
Secure External Calls	Checks Effects Interaction	1 Unsafe External Calls	1.1 Reentrancy	1.1.1 Unsafe Credit Transfer (CWE-841) 1.1.2 Unsafe System State Changes (CWE-841)
	Mutex	1 Unsafe External Calls	1.1 Reentrancy	1.1.1 Unsafe Credit Transfer (CWE-841) 1.1.2 Unsafe System State Changes (CWE-841)
	Pull Payment	1 Unsafe External Calls		
	Limit External Calls	1 Unsafe External Calls		
Secure Credit Transfer	Secure Ether Transfer	4 Erroneous Credit Transfer		
Secure Programming Practices	Randomness	5 Bad Programming Practices and Language Weaknesses	5.1 Bad Randomness	5.1 Bad Randomness (CWE-330)
Flow Control	Guard Check	6 Incorrect Control Flow		
	Escrow	6 Incorrect Control Flow		
Safe Arithmetic	SafeMath	7 Arithmetic Issues	7.1 Overflow and Underflow	7.1.1 Integer (CWE-191) 7.1.2 Integer Overflow (CWE-190)
Access Control	Authorization	8 Improper Access Control	8.1 Incorrect Authentication or Authorization	
	Time Constraint	8 Improper Access Control		
	Commit and Reveal	8 Improper Access Control	8.2 Improper Protection of Sensitive Data	
	Ownership	8 Improper Access Control	8.1 Incorrect Authentication or Authorization	8.1.2 Owner Manipulation (CWE-732)
	Access Restriction	8 Improper Access Control	8.1 Incorrect Authentication or Authorization	
	Blocklist	8 Improper Access Control		

Table 8 continued

Subcategory DPs	Security DPs	Category of Vulnerability	Vulnerability Subcategory	Vulnerability
	Dynamic Binding	8 Improper Access Control		
	Embedded Permission	8 Improper Access Control	8.1 Incorrect Authentication or Authorization	
	Selective Content Generation	8 Improper Access Control	8.2 Improper Protection of Sensitive Data	
	Tokenized Exchange	8 Improper Access Control		
	Single Authorization	8 Improper Access Control	8.1 Incorrect Authentication or Authorization	
	Stealth Address	8 Improper Access Control	8.1 Incorrect Authentication or Authorization	
	Authorized Spender	8 Improper Access Control	8.1 Incorrect Authentication or Authorization	
Integrity	Hash Integrity			
	File Comparison			
	Reverse Verifier			
Privacy	Multiple Registrations			
	Zero Knowledge Proofs			
	Key Sharding			
Risk Mitigation	Emergency Stop			
	Speed Bump			
	Rate Limit			
	Balance Limit			
	Multi-signature			
Auxiliary	Key Generation			
	Database Proxy			
	Hashed Timelock Contract			

Mitigation focuses on reducing security threats, while Secure External Calls addresses safe interactions with external contracts or systems.

– **Completeness of security design patterns in terms of smart contract vulnerability coverage:** Among the security design patterns identified in the previous section, we found five patterns explicitly targeting one or more known smart contract vulnerabilities. Together, these patterns address only 6 out of the 94 known vulnerabilities presented in OpenSCV. In addition, it is worthwhile mentioning that two OpenSCV families of vulnerabilities, namely "#3 Mishandled Events" and "#4 Gas Depletion" (Vidal et al. 2024), are not considered by any pattern. While this is understandable due to the overall low number of vulnerabilities addressed, it further highlights a significant gap in the availability of design solutions that can aid developers in preventing vulnerabilities. It is likely that families of related vulnerabilities may partially share solutions or that at least the insights gained from designing a solution for a certain vulnerability can help in designing solutions for related vulnerabilities. This is an aspect for which awareness must be raised.

In summary, the dynamic and evolving nature of blockchain technology means that new vulnerabilities continuously emerge, posing ongoing challenges in smart contract development. Despite the progress made, the ever-changing landscape of security threats necessitates ongoing research of security design patterns. Addressing these challenges is essential not only for advancing the theoretical framework of blockchain technology but also for ensuring the practical security and reliability of smart contract applications in real-world scenarios.

This work highlighted significant gaps where **further research is needed across various dimensions**. Here, we outline several potential directions for future research:

– The first research direction may be to target vulnerability detection tools. Such tools may help collect the different forms of security vulnerabilities and associated information to formalize the corresponding design patterns. Thus, establishing the appropriate *links between vulnerability detection tools and the vulnerability information that they collect* may be necessary for acknowledging a recurrent problem, collecting its characteristics, and contributing information that is essential for defining new design patterns.
– While our identification and categorization of security design patterns may serve as a foundation that allows for evolution, such evolution will only occur if research on security design patterns with direct connections to security vulnerabilities is carried out. Thus, a second research direction should be set on further *extending the current set of known security design patterns for smart contracts*.
– A third research direction may be directed towards the process of defining smart contract design patterns and may involve, for instance, the creation of a *collaborative smart contract patterns repository*. This would include researchers and practitioners collaborating in gathering information, identifying and proposing solutions, voting or selecting the best candidates for patterns, and iteratively reviewing design patterns. An up-to-date catalog of security design patterns is a valuable asset when the end goal is to improve the security of smart contracts.
– One of the main gaps identified in this work is indeed the very limited coverage of current design patterns regarding current smart contract vulnerabilities. While many current design patterns target more common risks such as reentrancy or integer overflows, other less widespread or more recent vulnerabilities are not addressed. Obviously, there may be cases of vulnerabilities for which a design pattern may not help, such as a wrong compiler version set on a smart contract (i.e., 5.8.2 Outdated Compiler Version in OpenSCV). Despite this, the large number of unaddressed vulnerabilities leads us to conclude that

addressing this gap should, in our view, be the key priority in advancing the state of the art in smart contract security design patterns.

7 Conclusion

This paper reviewed the state of the art in smart contract design patterns, with particular emphasis on security design patterns. We categorized the various design patterns and analyzed how well current security patterns avoid known security vulnerabilities. Results have shown that the security design patterns identified address a small number of security vulnerabilities (6 out of 94 in OpenSCV). In future work, we intend to understand how the design patterns catalog presented in this paper can be extended to avoid a broader range of vulnerabilities.

Acknowledgements This work is financed through national funds by FCT - Fundação para a Ciência e a Tecnologia, I.P., in the framework of the Project UIDB/00326/2025 and UIDP/00326/2025; by Project "Agenda Mobilizadora Sines Nexus". ref. No. 7113, supported by the Recovery and Resilience Plan (PRR) and by the European Funds Next Generation EU, following Notice No. 02/C05-i01/2022, Component 5 - Capitalization and Business Innovation - Mobilizing Agendas for Business Innovation; and by Foundation for Science and Technology (FCT), Grant Nr. 2024.05209.BD.

Funding Open access funding provided by FCT|FCCN (b-on).

Data Availability All data used to write this paper is available at Zenodo. Additional resources can be found at https://blockchain.dei.uc.pt/patterns.

Declarations

Conflicts of interests The authors declare that they have no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

Abreu AW, Coutinho EF (2020) A pattern adherence analysis to a blockchain web application. In: 2020 IEEE international conference on software architecture companion (ICSA-C), pp 103–109. https://doi.org/10.1109/ICSA-C50368.2020.00028

Al-Breiki H, Rehman MHU, Salah K, Svetinovic D (2020) Trustworthy blockchain oracles: review, comparison, and open research challenges. IEEE Access 8:85675–8568. https://doi.org/10.1109/ACCESS.2020.2992698

Amani S, Bégel M, Bortin M, Staples M (2018) Towards verifying ethereum smart contract bytecode in isabelle/hol. In: Proceedings of the 7th ACM SIGPLAN international conference on certified programs and proofs, pp 66–77

Antonino P, Ferreira J, Sampaio A, Roscoe AW (2022) Specification is law: safe creation and upgrade of ethereum smart contracts. In: Schlingloff BH, Chai M (eds) Software engineering and formal methods. Springer International Publishing, Cham, pp 227–243

Argañaraz M, Berón M, Pereira MJ, Henriques PR (2020) Detection of vulnerabilities in smart contracts specifications in ethereum platforms. In: 9th Symposium on Languages, Applications and Technologies

(SLATE 2020), Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, vol 83, pp 1–16,https://doi.org/10.4230/OASIcs.SLATE.2020.2

Bandara HD, Xu X, Weber I (2020) Patterns for blockchain data migration. In: Proceedings of the European conference on pattern languages of programs 2020, Association for Computing Machinery, New York, NY, USA, EuroPLoP '20,https://doi.org/10.1145/3424771.3424796

Bartoletti M, Pompianu L (2017) An empirical analysis of smart contracts: Platforms, applications, and design patterns. In: Financial Cryptography and Data Security, Springer International Publishing, Cham, pp 494–509. https://doi.org/10.48550/arXiv.2102.09810

Bartoletti M, Bracciali A, Lepore C, Scalas A, Zunino R (2020) A formal model of algorand smart contracts. https://doi.org/10.48550/arXiv.2009.12140 arxiv:2009.12140

Baryshev Y (2019) Design patterns security analysis for blockchain-based applications development with javascript and solidity

Bloo F (2018) Towards updatable smart contracts

Blum F, Severin B, Hettmer M, Hückinghaus P, Gruhn V (2020) Building hybrid dapps using blockchain tactics -the meta-transaction example. In: 2020 IEEE international conference on blockchain and cryptocurrency (ICBC), pp 1–5. https://doi.org/10.1109/ICBC48266.2020.9169423

Britten D, Sjöberg V, Reeves S (2021) Using coq to enforce the checks-effects-interactions pattern in deepsea smart contracts. In: FMBC 2021

Bui VC, Wen S, Yu J, Xia X, Haghighi MS, Xiang Y (2021) Evaluating upgradable smart contract. In: 2021 IEEE international conference on blockchain (Blockchain), pp 252–256. https://doi.org/10.1109/Blockchain53845.2021.00041

Chan E, Lesani M (2021) Brief announcement: Brokering with hashed timelock contracts is np-hard. In: Proceedings of the 2021 ACM symposium on principles of distributed computing, Association for Computing Machinery, New York, NY, USA, PODC'21, pp 199-202. https://doi.org/10.1145/3465084.3467952

Chatterjee A, Pitroda Y, Parmar M (2020) Dynamic role-based access control for decentralized applications. In: Chen Z, Cui L, Palanisamy B, Zhang LJ (eds) Blockchain - ICBC 2020. Springer International Publishing, Cham, pp 185–197

Chen H, Pendleton M, Njilla L, Xu S (2020) A survey on ethereum systems security: vulnerabilities, attacks, and defenses. ACM Comput Surv (CSUR) 53(3):1–43

Chen J, Xia X, Lo D, Grundy J, Luo X, Chen T (2022) Defining smart contract defects on ethereum. IEEE Trans Softw Eng 48(1):327–34. https://doi.org/10.1109/TSE.2020.2989002

Dika A, Nowostawski M (2018) Security vulnerabilities in ethereum smart contracts. In: 2018 IEEE international conference on internet of things (iThings) and IEEE green computing and communications (GreenCom) and IEEE cyber, physical and social computing (CPSCom) and IEEE smart data (SmartData), pp 955–962. https://doi.org/10.1109/Cybermatics_2018.2018.00182

Ebrahimi AM, Adams B, Oliva GA, Hassan AE (2024) A large-scale exploratory study on the proxy pattern in ethereum. Empir Softw Eng 29(4):1–51

Fahmideh M, Grundy J, Ahmad A, Shen J, Yan J, Mougouei D, Wang P, Ghose A, Gunawardana A, Aickelin U et al (2022) Engineering blockchain-based software systems: foundations, survey, and future directions. ACM Comput Surv 55(6):1–44

Ferreira Torres C (2022) From smart to secure contracts: Automated security assessment and improvement of ethereum smart contracts. PhD thesis, University of Luxembourg, Luxembourg

Gamma E, Helm R, Johnson R, Vlissides J (1993) Design patterns: abstraction and reuse of object-oriented design. In: ECOOP'93-Object-Oriented Programming: 7th European Conference Kaiserslautern, Germany, July 26–30, 1993 Proceedings 7, Springer, pp 406–431

Gao Z, Zhuang Z, Lin Y, Rui L, Yang Y, Zhao C, Mo Z (2021) Select-storage: a new oracle design pattern on blockchain. In: 2021 IEEE 20th international conference on trust, security and privacy in computing and communications (TrustCom), pp 1177–1184. https://doi.org/10.1109/TrustCom53373.2021.00159

Goldfeder S, Bonneau J, Gennaro R, Narayanan A (2017) Escrow protocols for cryptocurrencies: How to buy physical goods using bitcoin. In: Financial cryptography and data security: 21st international conference, FC 2017, Sliema, Malta, April 3-7, 2017, Revised Selected Papers 21, Springer, pp 321–339

Guida L, Daniel F (2019) Supporting reuse of smart contracts through service orientation and assisted development. In: 2019 IEEE International conference on decentralized applications and infrastructures (DAPPCON), pp 59–68. https://doi.org/10.1109/DAPPCON.2019.00017

Hickey T, Ju Q, Van Emden MH (2001) Interval arithmetic: from principles to implementation. J ACM 48(5):1038–1068. https://doi.org/10.1145/502102.502106

Hornyák O, Alkhoury GF (2020) Smart contracts in the automotive industry. In: Vehicle and automotive engineering, Springer, pp 148–157

Huang J, Kong L, Wang J, Chen G, Gao J, Huang G, Khan MK (2024) Secure data sharing over vehicular networks based on multi-sharding blockchain. ACM Trans Sensor Netw 20(2):1–23

Hunt J, Hunt J (2013) Gang of four design patterns. Patterns for Practical Reuse and Design, Scala Design Patterns, pp 135–136

Imeri A, Lamont J, Agoulmine N, Khadraoui D (2019) Model of dynamic smart contract for permissioned blockchains. In: PoEM (Forum), pp 1–15

Jurgelaitis M, čeponienė L, Butkienė R (2022) Solidity code generation from uml state machines in model-driven smart contract development. IEEE Access 10:33465–3348. https://doi.org/10.1109/ACCESS.2022.3162227

Kannengießer N, Lins S, Sander C, Winter K, Frey H, Sunyaev A (2022) Challenges and common solutions in smart contract development. IEEE Trans Softw Eng 48(11):4291–4318. https://doi.org/10.1109/TSE.2021.3116808

Khan SN, Loukil F, Ghedira-Guegan C, Benkhelifa E, Bani-Hani A (2021) Blockchain smart contracts: applications, challenges, and future trends. Peer Peer Network Appl 14:2901–2925

Kitchenham B (2004) Procedures for performing systematic reviews. UK, Keele Univ, Keele, p 33

Kovács AM, Seres IA (2024) Anonymity analysis of the umbra stealth address scheme on ethereum. Association for Computing Machinery, New York, NY, USA. https://doi.org/10.1145/3589335.3651963

Lamken D, Wagner T, Hoiss T, Seidenfad K, Hermann A, Kus M, Lechner U (2021) Design patterns and framework for blockchain integration in supply chains. In: 2021 IEEE international conference on blockchain and cryptocurrency (ICBC), pp 1–3. https://doi.org/10.1109/ICBC51069.2021.9461062

Li C, Palanisamy B, Xu R (2019) Scalable and privacy-preserving design of on/off-chain smart contracts. In: 2019 IEEE 35th international conference on data engineering workshops (ICDEW), pp 7–12. https://doi.org/10.1109/ICDEW.2019.00-43

Liu Y, Lu Q, Xu X, Zhu L, Yao H (2018) Applying design patterns in smart contracts. In: Chen S, Wang H, Zhang LJ (eds) Blockchain – ICBC 2018, Springer International Publishing, Cham, pp 92–106. https://doi.org/10.1007/978-3-319-94478-4_7

Liu Y, Lu Q, Paik HY, Xu X (2020) Design patterns for blockchain-based self-sovereign identity. Proc Eur Conf Pattern Lang Prog 2020:1–14. https://doi.org/10.1145/3424771.3424802

Liu Y, Lu Q, Paik HY, Xu X, Chen S, Zhu L (2020) Design pattern as a service for blockchain-based self-sovereign identity. IEEE Softw 37(5):30–3. https://doi.org/10.1109/MS.2020.2992783

Lopez Marin JC (2022) Upgradeable smart contracts design patterns for dapps architectures

Lu Q, Xu X, Liu Y, Zhang W (2018) Design pattern as a service for blockchain applications. In: 2018 IEEE international conference on data mining workshops (ICDMW), pp 128–135. https://doi.org/10.1109/ICDMW.2018.00025

Lu Q, Xu X, Bandara HD, Chen S, Zhu L (2022) Patterns for blockchain-based payment applications. In: Proceedings of the 26th european conference on pattern languages of programs, Association for Computing Machinery, New York, NY, USA, EuroPLoP '21. https://doi.org/10.1145/3489449.3490006

Marchesi L, Marchesi M, Destefanis G, Barabino G, Tigano D (2020) Design patterns for gas optimization in ethereum. In: 2020 IEEE international workshop on blockchain oriented software engineering (IWBOSE), pp 9–15. https://doi.org/10.1109/IWBOSE50093.2020.9050163

Mavridou A, Laszka A (2018) Tool demonstration: fsolidm for designing secure ethereum smart contracts. In: Bauer L, Küsters R (eds) Principles of Security and Trust. Springer International Publishing, Cham, pp 270–277

Van der Meyden R, Maher MJ (2021) Architecture for smart safe contracts. In: 2021 3rd Conference on blockchain research & applications for innovative networks and services (BRAINS) pp 145–148. https://doi.org/10.1109/BRAINS52497.2021.9569831

Monika Bhatia R, Jain A, Singh B (2022) Hash time locked contract based asset exchange solution for probabilistic public blockchains. Cluster Comput 25(6):4189–4201

Moura TAP (2020) Design patterns for ethereum smart contracts

Mühlberger R, Bachhofner S, Castelló Ferrer E, Di Ciccio C, Weber I, Wöhrer M, Zdun U (2020) Foundational oracle patterns: Connecting blockchain to the off-chain world. In: Business process management: blockchain and robotic process automation forum: BPM 2020 Blockchain and RPA Forum, Seville, Spain, September 13–18, 2020, Proceedings 18, Springer, pp 35–51

N'Da A, Matalonga S, Dahal K (2020) Characterizing the cost of introducing secure programming patterns and practices in ethereum. In: WorldCist'20 - 8th world conference on information systems and technologies, Springer, pp 25–3https://doi.org/10.1007/978-3-030-45691-7_3

Ordoñez-Guerrero AC, Muñoz-Garzon JD, Roberto Dulce Villarreal E, Bandi A, Ariel Hurtado J (2022) Blockchain architectural concerns: a systematic mapping study. In: 2022 IEEE 19th International conference on software architecture companion (ICSA-C), pp 183–192. https://doi.org/10.1109/ICSA-C54293.2022.00043

Oruç O, Aßmann U, Troshani A (2022) Context-based role object pattern with on-chain smart contract programming. In: Konferenzband zum Scientific Track der Blockchain Autumn School 2022, Hochschule Mittweida, 2, pp 42–50

Parmar M, Kaur HJ (2021) Comparative analysis of secured hash algorithms for blockchain technology and internet of things. Int J Adv Comput Sci Appl 12(3)

Penelova M (2021) Access control models. Cybern Inf Technol 21(4):77–104

Raab R (2021) Towards platform-agnostic smart contracts

Rajasekar V, Sondhi S, Saad S, Mohammed S (2020) Emerging design patterns for blockchain applications. In: Proceedings of the 15th international conference on software technologies - ICSOFT, INSTICC, SciTePress, pp 242–249. https://doi.org/10.5220/0009892702420249

Rameder H, Di Angelo M, Salzer G (2022) Review of automated vulnerability analysis of smart contracts on ethereum. Front Blockchain 5:814977

Ren M, Ma F, Yin Z, Fu Y, Li H, Chang W, Jiang Y (2021) Making smart contract development more secure and easier. In: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp 1360–1370

Sadaf A, Golzari A, Ivaki N, Laranjeiro N (2024) A Systematic Review on Smart Contracts Security Design Patterns - Supplementary Material.https://doi.org/10.5281/zenodo.13170977

Samarati P, De Vimercati SC (2000) Access control: Policies, models, and mechanisms. In: International school on foundations of security analysis and design, Springer, pp 137–196

Samreen NF, Alalfi MH (2023) An empirical study on the complexity, security and maintainability of ethereum-based decentralized applications (dapps). Blockchain: Res Appl 4(2):100120. https://doi.org/10.1016/j.bcra.2022.100120

Sayeed S, Marco-Gisbert H, Caira T (2020) Smart contract: attacks and protections. IEEE Access 8:24416–2442. https://doi.org/10.1109/ACCESS.2020.2970495

Singh A, Parizi RM, Zhang Q, Choo KKR, Dehghantanha A (2020) Blockchain smart contracts formalization: Approaches and challenges to address vulnerabilities. Comput Secur. https://doi.org/10.1016/j.cose.2019.101654 https://www.sciencedirect.com/science/article/pii/S0167404818310927

Six N, Correa-Restrepo C, Herbaut N, Salinesi C (2022a) An ontology for software patterns: Application to blockchain-based software development. In: International conference on enterprise design, operations, and computing, Springer, pp 284–299

Six N, Herbaut N, Salinesi C (2022b) Blockchain software patterns for the design of decentralized applications: a systematic literature review. vol 3, p 10006. https://doi.org/10.1016/j.bcra.2022.100061

Songsom N, Werapun W, Suaboot J, Rattanavipanon N (2022) The swc-based security analysis tool for smart contract vulnerability detection. In: 2022 6th International conference on information technology (InCIT), pp 74–77. https://doi.org/10.1109/InCIT56086.2022.10067759

Sun J, Huang S, Zheng C, Wang T, Zong C, Hui Z (2022) Mutation testing for integer overflow in ethereum smart contracts. Tsinghua Sci Technol 27(1):27–40. https://doi.org/10.26599/TST.2020.9010036

Sun X, Yu FR, Zhang P, Sun Z, Xie W, Peng X (2021) A survey on zero-knowledge proof in blockchain. IEEE Netw 35(4):198–205. https://doi.org/10.1109/MNET.011.2000473

Tan B, Mariano B, Lahiri SK, Dillig I, Feng Y (2022) Soltype: refinement types for arithmetic overflow in solidity. Proc ACM Program Lang 6(POPL):1–29

Taylor A, Kugler A, Marella PB, Dagher GG (2022) Vigilrx: a scalable and interoperable prescription management system using blockchain. IEEE Access 10:25973–2598. https://doi.org/10.1109/ACCESS.2022.3156015

Töberg JP, Schiffl J, Reiche F, Beckert B, Heinrich R, Reussner R (2022) Modeling and enforcing access control policies for smart contracts. In: 2022 IEEE international conference on decentralized applications and infrastructures (DAPPS), pp 38–47. https://doi.org/10.1109/DAPPS55202.2022.00013

Thirulokachander VR (2021) Impact of design patterns on code quality in blockchain-based applications

Trojanowska N, Kedziora M, Hanif M, Song H (2020) Secure decentralized application development of blockchain-based games. In: 2020 IEEE 39th international performance computing and communications conference (IPCCC), pp 1–8. https://doi.org/10.1109/IPCCC50635.2020.9391556

Verdonck M (2019) Demonstrating the importance of well-defined design patterns in smart contract development. In: Proceedings of the international workshop on value modeling and business ontologies (VMBO 2019), pp 1–5

Vidal FR, Ivaki N, Laranjeiro N (2024) OpenSCV: an open hierarchical taxonomy for smart contract vulnerabilities. Empir Softw Eng 29(4):101. https://doi.org/10.1007/s10664-024-10446-8

Volland F (2019) Solidity patterns. https://github.com/fravoll/solidity-patterns/, Accessed 01-March-2020

Wang G, Shi ZJ, Nixon M, Han S (2019a) Sok: Sharding on blockchain. In: Proceedings of the 1st ACM conference on advances in financial technologies, pp 41–61

Wang Z, Yang L, Wang Q, Liu D, Xu Z, Liu S (2019b) Artchain: Blockchain-enabled platform for art marketplace. In: 2019 IEEE international conference on blockchain (Blockchain), pp 447–454. https://doi.org/10.1109/Blockchain.2019.00068

Wang Z, Jin H, Dai W, Choo KKR, Zou D (2021) Ethereum smart contract security research: survey and future research opportunities. Front Comput Sci 15:1–18

Wedyan F, Abufakher S (2020) Impact of design patterns on software quality: a systematic literature review. IET Softw 14(1):1–17

Wei Q, Li B, Chang W, Jia Z, Shen Z, Shao Z (2022) A survey of blockchain data management systems. ACM Trans Embedded Comput Syst (TECS) 21(3):1–28

Wessling F, Ehmke C, Meyer O, Gruhn V (2019) Towards blockchain tactics: Building hybrid decentralized software architectures. In: 2019 IEEE international conference on software architecture companion (ICSA-C), pp 234–237. https://doi.org/10.1109/ICSA-C.2019.00048

Wöhrer M, Zdun U (2018) Design patterns for smart contracts in the ethereum ecosystem. In: 2018 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData), pp 1513–1520. https://doi.org/10.1109/Cybermatics_2018.2018.00255

Wöhrer M, Zdun U (2020) Domain specific language for smart contract development. In: 2020 IEEE international conference on blockchain and cryptocurrency (ICBC), pp 1–9. https://doi.org/10.1109/ICBC48266.2020.9169399

William S (2008) Operating systems: internals and design principles (sisth edition)

Wohrer M, Zdun U (2018) Smart contracts: security patterns in the ethereum ecosystem and solidity. In: 2018 International workshop on blockchain oriented software engineering (IWBOSE), pp 2–8. https://doi.org/10.1109/IWBOSE.2018.8327565

Wong D, Hemmel M (2018) DASP Top 10 of 2018. https://www.dasp.co, Accessed: 28-Sept-2023

Worley CR, Skjellum A (2019) Opportunities, challenges, and future extensions for smart-contract design patterns. In: Abramowicz W, Paschke A (eds) Business information systems workshops, Springer International Publishing, Cham, pp 264–276. https://doi.org/10.1007/978-3-030-04849-5_24

Xu X, Pautasso C, Zhu L, Lu Q, Weber I (2018) A pattern collection for blockchain-based applications. In: Proceedings of the 23rd European conference on pattern languages of programs, pp 1–20. https://doi.org/10.1109/ICDMW.2018.00025

Xu X, Bandara HMND, Lu Q, Zhang D, Zhu L (2020) Understanding and handling blockchain uncertainties. In: Chen Z, Cui L, Palanisamy B, Zhang LJ (eds) Blockchain - ICBC 2020. Springer International Publishing, Cham, pp 108–124

Xu X, Dilum Bandara H, Lu Q, Weber I, Bass L, Zhu L (2021) A decision model for choosing patterns in blockchain-based applications. In: 2021 IEEE 18th international conference on software architecture (ICSA), pp 47–57. https://doi.org/10.1109/ICSA51549.2021.00013

Yang X (2021) Blockchain-based supply chain finance design pattern. In: 2021 13th International conference on intelligent human-machine systems and cybernetics (IHMSC), pp 200–203. https://doi.org/10.1109/IHMSC52134.2021.00053

Zecchini M, Bracciali A, Chatzigiannakis I, Vitaletti A (2019) On refining design patterns for smart contracts. In: European conference on parallel processing, Springer, pp 228–239

Zhang P, Schmidt DC, White J (2020) A pattern sequence for designing blockchain-based healthcare information technology systems. https://doi.org/10.48550/arXiv.2010.01172

Zhang R, Xue R, Liu L (2019) Security and privacy on blockchain. ACM Comput Surv (CSUR) 52(3):1–34

Zheng Z, Xie S, Dai HN, Chen W, Chen X, Weng J, Imran M (2020) An overview on smart contracts: Challenges, advances and platforms. Future Gener Comput Syst 105:475–491. https://doi.org/10.1016/j.future.2019.12.019 https://www.sciencedirect.com/science/article/pii/S0167739X19316280

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.